



Chat-First Data Analytics

Ulrich Matter

Bern University of Applied Sciences

University of St. Gallen

Abstract

Using natural language to drive data analysis is an old idea, but large language models (LLMs) have made it suddenly ubiquitous. A widely observed and rather concerning workflow has users upload a dataset to ChatGPT and ask for an analysis; the model generates and executes code on a remote virtual machine, producing results that are difficult to verify, difficult to reproduce, and incompatible with data-privacy requirements. This paper describes an alternative approach called *chat-first data analytics*, built on three principles: separate interpretation from computation, pre-validate methods rather than generate code, and transmit only metadata—not data. The language model interprets intent and selects tools; pre-implemented, validated algorithms produce results. **prompt2analytics** implements this approach as an econometrics and data analytics library in pure Rust, exposed through the Model Context Protocol (MCP) for integration with language models. The library covers regression, panel data, instrumental variables, difference-in-differences, discrete choice, time series, and over 200 additional methods—all validated against established R packages. Benchmarks show a median speedup of $4\times$ over corresponding R implementations at $n = 100,000$. Evaluation shows that tool selection achieves 90–100% accuracy on curated prompts. Local deployment via Ollama enables complete data isolation on consumer hardware.

Keywords: chat-first analytics, natural language interface, econometrics, Rust, MCP, local LLM, data privacy.

1. Introduction

The idea of talking to a computer to analyze data is not new. Natural language interfaces to databases date to the 1970s (Woods 1973; Codd 1974), and every generation of statistical software has tried to lower the barrier between analytical intent and software commands. What *is* new is that the idea suddenly works well enough to be everywhere. Since late 2022, millions of users have been uploading datasets to ChatGPT and typing requests like “compare

last quarter’s sales by region” (Hu 2023). The conversational interface to data analysis has gone from research curiosity to mass-market product in under two years.

The results are concerning. The model generates a Python script and executes it on a remote virtual machine—the user’s data now resides on a third-party server they do not control. Even when users copy generated code back to their own machine, the problems persist: generated code may compute the wrong quantity without raising an error (Ludwig, Mullainathan, and Rambachan 2025), and the same prompt yields different code as models update—making results irreproducible. Section 2 examines these failure modes in detail.

The core idea—natural language as interface—is sound. The architecture is not. These problems are not inherent to conversational data analysis; they arise from a specific design choice: delegating computation to generated code. This paper describes an alternative approach we call *chat-first data analytics*, built on three principles:

1. **Interpretation and computation are separated.** The language model selects methods and extracts parameters from natural language; pre-implemented, validated algorithms perform the actual computation.
2. **Methods are pre-validated, not generated.** Correctness is established once, through systematic validation against reference implementations, rather than hoped for anew with each request.
3. **Data stays local; only structure travels.** The language model receives column names, data types, and summary statistics—not the data itself. For complete isolation, the language model can run locally too.

These principles address each concern directly. Correctness follows from using validated implementations rather than generated code. Privacy is protected by keeping data on the user’s machine. Reproducibility is achieved because the same tool invocation produces identical results—the stochastic element of the language model affects only interpretation, not computation.

This paper presents **prompt2analytics** (**p2a**), an implementation of chat-first data analytics. The software includes an econometrics and data analytics library in pure Rust, exposed through the Model Context Protocol (MCP) for integration with language models. The implementation covers ordinary least squares (OLS) with robust standard errors, panel data models with fixed effects (FE) and random effects (RE), instrumental variables (IV), difference-in-differences (DiD), discrete choice, time series, and over 200 additional methods—all validated against established R packages.

prompt2analytics is available as open-source software under the MIT license at <https://github.com/umatter/prompt2analytics>. Supplementary materials—including the 87 evaluation test cases, benchmark scripts for both R and Rust, and validation test suites—are located in the repository’s **paper/** directory. Appendix F provides reproduction instructions.

The remainder of the paper is organized as follows. Section 2 motivates chat-first analytics by examining the failure modes of current approaches and situating the three principles within existing work. Section 3 describes the software implementation. Section 4 demonstrates the system through worked examples including realistic failure cases. Section 5 evaluates tool selection accuracy and numerical correctness. Section 6 presents performance benchmarks and local deployment configurations. Section 7 discusses limitations and future directions.

2. Defining chat-first analytics

The introduction identified three concerns with using language models for data analysis: correctness, privacy, and reproducibility. This section examines these failure modes more carefully, then states the three principles that define chat-first analytics.

2.1. How LLM-assisted analysis goes wrong

The appeal of typing “regress wages on education with robust standard errors” into ChatGPT is obvious. The problems are less visible. They fall into four categories.

First, LLM-generated code often runs without error but computes the wrong quantity. Requesting “clustered standard errors” might yield heteroskedasticity-robust errors instead. A panel model might omit entity fixed effects. A difference-in-differences specification might use the wrong interaction term. [Ludwig *et al.* \(2025\)](#) document these risks systematically: the code executes, returns plausible-looking output, and the user has no way to detect the error without the expertise that would have let them write the code themselves.

Second, code-generation systems require the data. When a user asks ChatGPT to analyze a dataset, the data is uploaded to a third-party server with unclear retention policies. For survey responses, health records, personnel data, or any dataset subject to privacy regulations, this may be unacceptable. Even when providers promise not to retain data, the transmission itself may violate institutional or regulatory requirements.

Third, the same prompt sent to the same model on different days may produce different code. Models are updated, fine-tuned, and retrained. The generated Python script that worked in January may not be what the model produces in March. Conversational sessions are ephemeral—unless the user manually saves the generated code, the analysis cannot be reproduced. This contradicts a basic requirement of scientific work.

Fourth, when a system generates 30 lines of Python to answer a statistical question, understanding what was computed requires reading and auditing those 30 lines of code. The user who could not write the code cannot audit it either. This problem extends beyond research: enterprise text-to-SQL deployments struggle with the same issue—ambiguous questions admit multiple valid interpretations, and users cannot verify which was executed ([Renggli, Ilyas, and Rekatsinas 2025](#)).

These four problems are distinct but related. Silent errors arise because the language model controls both *what* to compute and *how* to compute it. Privacy violations arise because computation and data must be co-located on the provider’s servers. Reproducibility failures arise because the generated code is a function of the model’s current state. Auditability failures arise because the computational logic is opaque—buried in generated code rather than invoked from a known, inspectable implementation. Systematic error rates for LLM-generated statistical code are not yet well established—[Ludwig *et al.* \(2025\)](#) document the *types* of errors that arise, but base rates depend on the model, the prompt, and the statistical method. The motivation for the architecture described below rests on the existence and nature of these failure modes, not on their precise frequency.

2.2. Three principles

Chat-first data analytics addresses these problems through three principles. “Chat-first”

means that natural language conversation is the designed-for entry point: tool schemas are annotated for LLM consumption, error messages are phrased for conversational context, and default parameters are chosen so that typical requests need minimal specification. It does not mean chat-only—**prompt2analytics** also provides a CLI that invokes the same methods directly (Section 3), and every tool call maps to a public Rust function usable without any LLM—but the conversational interface is the primary design target. The first two principles are definitional; the third is a design choice that we consider essential.

Principle 1: Separate interpretation from computation

The language model decides *what* to compute; validated software decides *how*. When a user asks to “estimate the effect of education on wages, controlling for experience, with robust standard errors,” the language model’s job is to determine that this requires OLS with HC1 standard errors, identify the column names, and generate a structured tool call. The actual matrix algebra—forming $(\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{y}$, computing the HC1 sandwich estimator—is performed by pre-implemented code that the language model cannot modify.

This division of labor plays to each component’s strengths. Language models are good at understanding intent, resolving ambiguity, and mapping natural language to structured specifications. They are unreliable at generating correct numerical code. Separation removes the failure mode where the model produces code that runs but computes the wrong quantity, because the model never produces code at all. Wrong parameter extraction remains possible (Section 5), but the error surface is smaller: the model selects from a fixed schema rather than generating arbitrary code.

The idea has precedent in text-to-SQL systems (Yu, Zhang, Yang, Yasunaga, Wang, Li, Ma, Li, Yao, Roman, Zhang, and Radev 2018; Li, Luo, Chai, Li, and Tang 2024), where a language model translates natural language into structured database queries executed by a SQL engine. We extend this pattern from data retrieval to statistical computation: from text-to-SQL to text-to-statistics.

Principle 2: Pre-validate methods, not generated code

Correctness is established once, through systematic validation against reference implementations, rather than hoped for anew with each request. When **prompt2analytics** runs OLS, it uses the same Rust function every time—a function validated against R’s `lm()` on the Longley dataset to 10^{-8} precision (Section 5). The user need not audit the implementation on each invocation, any more than they audit R’s source code when calling `lm()`.

This principle also solves the reproducibility problem. The same method invocation with the same data produces identical results, regardless of when it is run or which language model is used. The stochastic element of the language model affects only interpretation (which tool to call, how to explain the output), never computation. Two users who phrase the same analysis differently may get different tool calls—but once the tool is selected and parameters specified, the result is deterministic.

Principle 3: Keep data local; transmit only structure

The language model receives column names, data types, and summary statistics—not the data itself. A dataset of 100,000 salary records is represented to the model as metadata:

“columns: employee_id (int), salary (float, range 28000–185000), department (string, 12 unique values), years_experience (float, range 0.5–35.2).” Statistical computations execute on the user’s machine; only aggregated results (coefficients, standard errors, test statistics) pass through the language model.

This is not definitionally required for chat-first analytics—a cloud-based system satisfying Principles 1 and 2 would still qualify. But we consider data locality essential for the use cases that motivate this work. Researchers analyzing survey data, firms analyzing personnel records, and analysts working under privacy regulations need architectures where data isolation is guaranteed by design, not promised by policy.

For complete isolation, the language model itself can run locally. Open-source models deployed via Ollama (Ollama Inc. 2024) eliminate all external network communication. Our evaluation (Section 5) shows that even 3-billion-parameter models achieve over 90% tool selection accuracy, making fully local deployment practical on consumer hardware.

Together, the three principles address the four failure modes identified above. Separation and pre-validation eliminate silent computational errors: the model never generates code, and the code it invokes is validated once against reference implementations. Pre-validation also ensures reproducibility, because the same method invocation with the same data always produces the same result. Data locality protects privacy by keeping raw data on the user’s machine. Auditability improves on all three fronts—the user inspects a structured tool call rather than 30 lines of generated code.

2.3. Relationship to existing ideas

Chat-first analytics applies established techniques to a new domain. Tool-augmented language models (Schick, Dwivedi-Yu, Dessì, Raileanu, Lomeli, Zettlemoyer, Cancedda, and Scialom 2023; Yao, Zhao, Yu, Du, Shafran, Narasimhan, and Cao 2023) can reliably select and invoke external tools when provided with appropriate schemas (Qu, Dai, Wei, Xu, Cai, Wang, Yin, and Wen 2025; Liu, Huang, Zeng, Hao, Yu, Li, Wang, Gan, Liu, Yu, Wang, Wang, Ning, Hou, Wang, Wu, Wang, Liu, Wang, Tang, Tu, Shang, Jiang, Tang, Lian, Liu, and Chen 2025). The Model Context Protocol (MCP) (Anthropic 2024) standardizes how language models discover and invoke tools, providing the communication layer between interpretation and computation.¹

The interface innovation has precedents in four strands of work. First, *formula notation and domain-specific languages*: the Wilkinson-Rogers formula notation (Wilkinson and Rogers 1973) simplified model specification in the 1970s; Chambers and Hastie (1992) extended it in S and R (Chambers 1998); and probabilistic programming languages like Stan (Carpenter, Gelman, Hoffman, Lee, Goodrich, Betancourt, Brubaker, Guo, Li, and Riddell 2017) apply the DSL pattern to Bayesian inference. Chat-first analytics replaces formal notation with natural language, trading precision for accessibility. Second, *natural language interfaces to data systems*: LUNAR (Woods 1973) demonstrated feasibility for database querying in the 1970s, and the text-to-SQL paradigm—driven by benchmarks like Spider (Yu *et al.* 2018)—has

¹MCP focuses on model-to-tool integration. Complementary protocols address agent-to-agent communication: Google’s Agent2Agent (A2A) protocol (Google 2025) and IBM’s Agent Communication Protocol (ACP) (IBM Research 2025), which merged under Linux Foundation governance in 2025. For single-agent tool invocation—the pattern used here—MCP is the dominant standard, with adoption by OpenAI, Microsoft, and Google alongside Anthropic.

produced substantial improvements in translating natural language to structured queries (Li *et al.* 2024). We extend this pattern from data retrieval to statistical computation. Third, *automated machine learning*: Auto-sklearn (Feurer, Klein, Eggenberger, Springenberg, Blum, and Hutter 2015) and H2O AutoML (LeDell and Poirier 2020) automate model selection through algorithmic search, and recent work integrates LLMs into the pipeline (Hollmann, Müller, and Hutter 2024). AutoML optimizes for prediction; chat-first analytics interprets user intent across prediction, causal inference, hypothesis testing, and description. Fourth, *computational econometrics* has established standards for numerical accuracy (McCullough 1997; McCullough and Vinod 1999; Davidson and MacKinnon 2004), which motivate the validation approach described in Section 5.

Several systems have explored LLM-assisted data analysis, but with a code-generation architecture. Hong, Lin, Liu, Liu, Wu, Li, Chen, Zhang, Wang, Zhang, Zhang, Yang, Zhuge, Guo, Zhou, Tao, Wang, Tang, Lu, Zheng, Liang, Fei, Cheng, Xu, and Wu (2025) present a “Data Interpreter” that generates and executes Python code. Chen, Shen, and Ma (2025) develop an “Econometrics AI Agent” that augments code generation with domain knowledge. Sun, Han, Jiang, Qi, Sun, Yuan, and Huang (2025) describe an agent that orchestrates generated code across analysis stages. All three generate code; none pre-validate implementations or enforce data locality. Table 1 summarizes these differences.

Hybrid designs are possible. A system could generate code for bespoke analyses while routing standard requests to pre-validated tools, or could verify generated code through static analysis before execution. **prompt2analytics** deliberately occupies one end of this spectrum: all computation is pre-validated, with no code generation at all. This sacrifices flexibility (analyses outside the method library are unavailable) but eliminates an entire class of failure modes. Whether hybrid designs can achieve a better trade-off is an open question.

Table 1: Comparison with code-generation approaches

	p2a	Econ. AI	Data Interpr.	LAMBDA
LLM role	Tool selection	Enhanced code gen	Code gen + exec	Agent orchestration
Computation	Pre-validated	LLM-generated	LLM-generated	LLM-generated
Local execution	Yes (Ollama)	No	No	No
Validated	Yes (vs. R)	No	No	No
Method flexibility	Fixed library (~200 methods)	Any R method	Any Python method	Any Python method
Extensibility	Requires Rust impl.	Extends with model	Extends with model	Extends with model

Note: Econ. AI = Econometrics AI Agent (Chen *et al.* 2025); Data Interpr. = Data Interpreter (Hong *et al.* 2025); LAMBDA (Sun *et al.* 2025). The trade-off: code-generation systems offer unlimited method flexibility at the cost of unvalidated computation; **p2a** constrains coverage to ensure correctness.

Two limitations of the approach are fundamental, not fixable by better engineering. First, statistical expertise is still required: chat-first analytics assists with method selection and execution, not with research design or causal identification. A user who requests “estimate the effect of X on Y” receives a regression, but the system does not validate whether a causal interpretation is warranted. LLM explanations may echo causal language from the user’s prompt even when the analysis provides only associational evidence. Users remain

responsible for ensuring that their identification strategy supports their claims.

Second, method selection is imperfect. Tool selection works well on curated prompts (Section 5), but users should verify that the system understood their request correctly—particularly for ambiguous specifications where multiple methods could apply. Users should consult the accuracy results before interpreting the curated examples in Section 4.

Third, pre-validation trades coverage for correctness. When a user requests a method outside the implemented set—Bayesian inference, structural equation modeling, or a custom estimator—the LLM may silently map the request to the closest available alternative rather than reporting unavailability. This is the symmetric failure mode to code generation’s silent errors: instead of running wrong code, the system runs the wrong method. The risk is mitigated by the structured tool schema (which makes the selected method visible to the user) and by the LLM’s ability to explain substitutions, but users must still verify that the method matches their intent.

These limitations mean that chat-first analytics is a useful tool, not an autonomous analyst. Users should verify tool selection, check extracted parameters, and apply domain knowledge to interpret results—the same discipline required when using any statistical software, with the added need to verify that the system understood the request correctly.

3. The software: **prompt2analytics** (p2a)

prompt2analytics implements the three principles defined in Section 2. This section describes the software architecture, the implemented methods, and the communication flow that connects natural language requests to statistical computation.

3.1. Architecture

The software is organized as a Rust workspace with four crates (Figure 1).

The core analytics library, **p2a-core**, implements all statistical methods in pure Rust with no external econometrics dependencies; all estimators are implemented from first principles using **ndarray** for matrix operations and **faer** for numerically stable linear algebra. Implementations follow standard textbook algorithms: OLS and robust standard errors follow Wooldridge (2010) and Cameron and Trivedi (2005); panel data methods follow Baltagi (2013); time series methods follow Hamilton (1994); maximum likelihood for discrete choice uses Newton-Raphson as described in Greene (2018). An optional `cuda` feature flag enables GPU acceleration for compute-intensive operations on NVIDIA hardware (Section 6). Three user-facing crates provide different access paths: **p2a-cli** produces a standalone **p2a** binary with hierarchical subcommands organized by analytical domain; **p2a-mcp** runs a Model Context Protocol server exposing tools for LLM integration via stdio, HTTP, or WebSocket transports; and **p2a-dioxus** provides a web application using Dioxus that compiles to WebAssembly for browser deployment. All three depend on **p2a-core**, ensuring identical numerical results regardless of the interface used.

The simplest way to use **prompt2analytics** is through the CLI:

```
# Load data and run OLS with robust standard errors
p2a data load wages.csv --name wages
p2a reg ols wages -y salary -x education experience --robust hc1
```

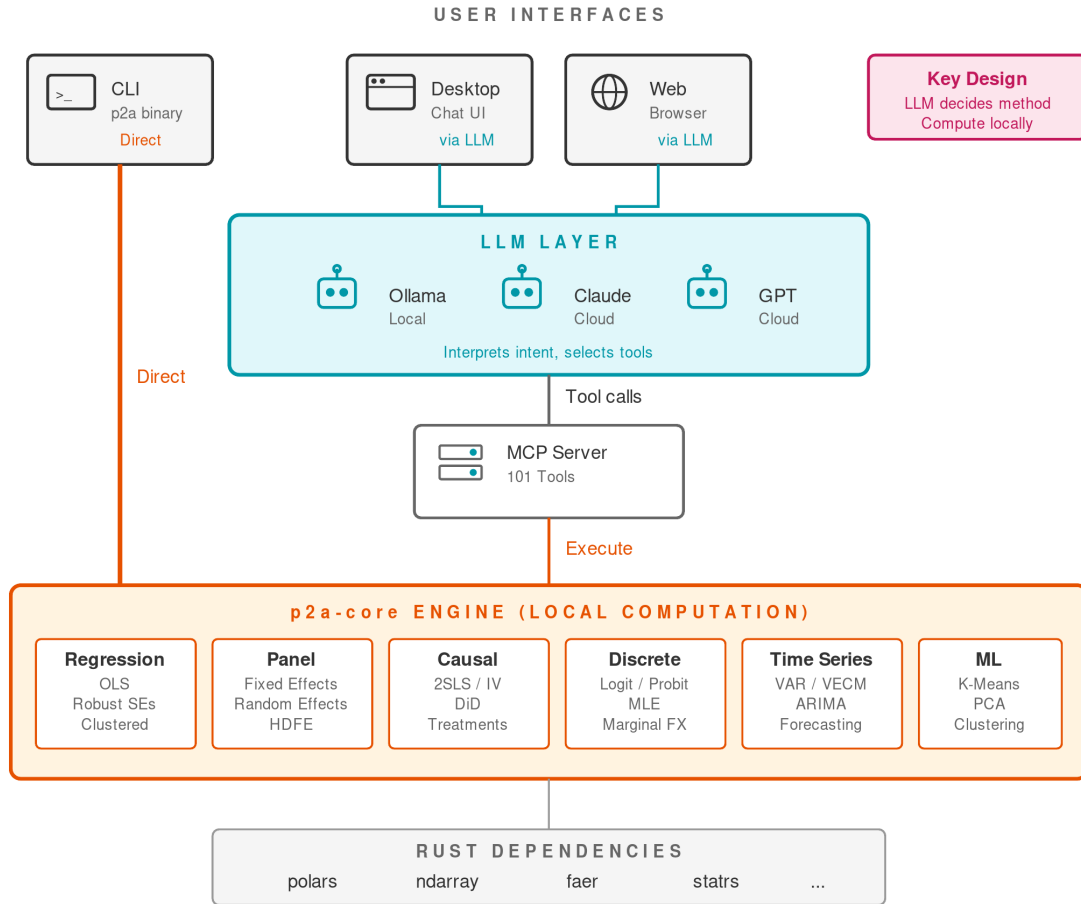


Figure 1: Software architecture of **prompt2analytics**. The CLI accesses the analytics engine directly (orange); chat interfaces route through the LLM layer and MCP server (cyan). All computation occurs locally in **p2a-core**.

For LLM-mediated access, configure the MCP server and type in natural language:

User: Load wages.csv and regress salary on education and experience with robust standard errors.

Both pathways invoke the same Rust implementation and produce identical numerical results.

3.2. How the principles work in practice

The communication flow (Figure 2) shows how Principle 1 (separation) operates concretely:

1. The user submits a natural language query.
2. The LLM interprets the request and generates a structured tool call with parameters—e.g., `regression_ols` with `y_var="salary"`, `robust="hc1"`, and the relevant column names.
3. The MCP server routes the call to the corresponding Rust method in **p2a-core**.
4. The core library performs computation locally.
5. Results (coefficients, standard errors, test statistics) are serialized to JSON and returned through the MCP server.
6. The LLM receives the results and formulates a natural language explanation.

The LLM never sees raw data. It receives metadata at dataset load time (column names, types, row counts) and aggregated results after computation. Data preview tools (`head`, `sample`) do transmit sample rows; for complete isolation, local LLM deployment via Ollama eliminates all external transmission.

Each tool returns its full result as serialized text: for regression this includes coefficients, standard errors, test statistics, R^2 , F -statistic, and information criteria (AIC, BIC); for discrete choice, log-likelihood and pseudo- R^2 ; for time series, parameter estimates and residual diagnostics. The LLM receives all of these and decides what to present based on the conversational context—typically highlighting coefficients and significance for a first pass, and surfacing fit measures or diagnostics when the user asks follow-up questions or when values are anomalous.

Each tool invokes a fixed Rust function validated against reference R implementations (Section 5). For linear regression, a `LinearEstimator` trait provides a consistent interface:

```
pub trait LinearEstimator {
    fn coefficients(&self) -> &Array1<f64>;
    fn std_errors(&self) -> &Array1<f64>;
    fn t_values(&self) -> Array1<f64>;    // default impl
    fn p_values(&self) -> Array1<f64>;    // default impl
    fn residuals(&self) -> Array1<f64>;
    fn n_obs(&self) -> usize;
    fn df(&self) -> usize;
}
```

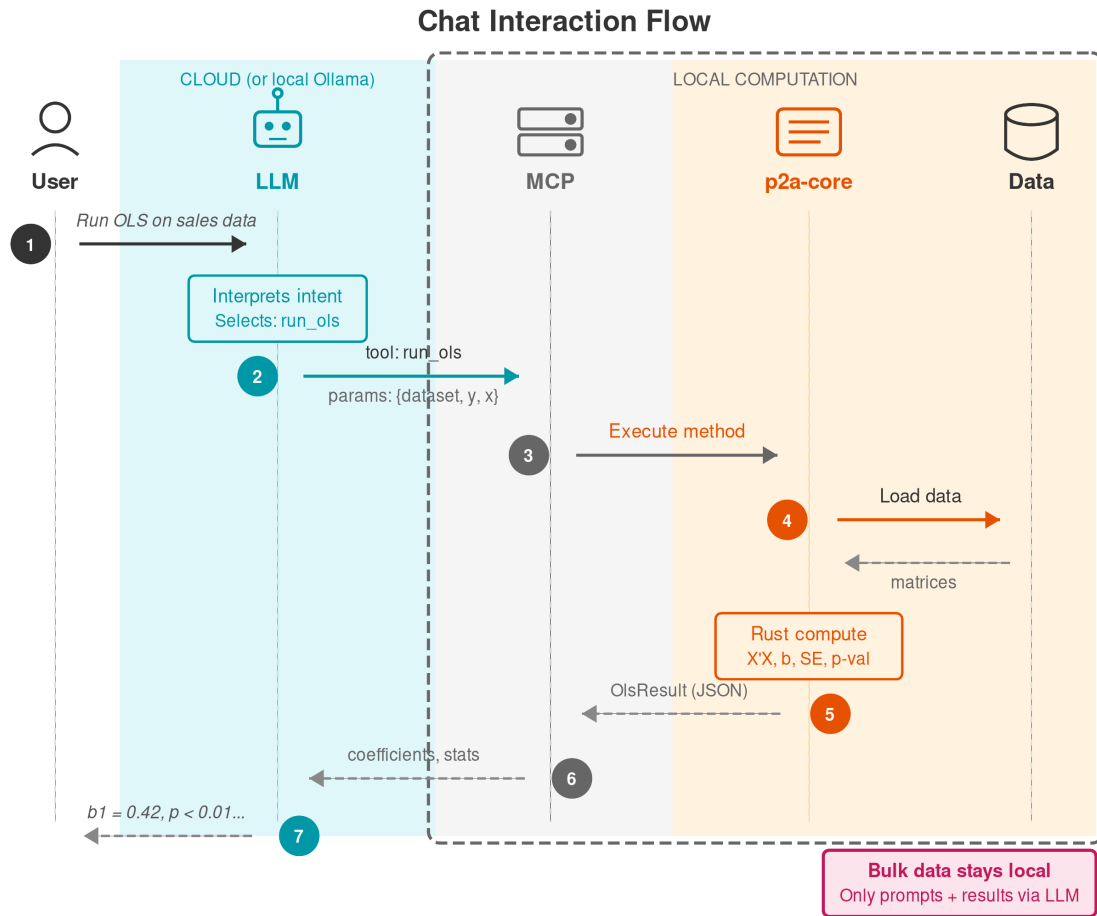


Figure 2: Interaction flow from user request to result. The LLM generates a structured tool call; the MCP server executes the Rust method; results flow back for explanation. Data processing (shaded) stays local.

Implementors provide core results; derived statistics (t -values, p -values, confidence intervals) are computed by default implementations, enforced at compile time. Other method families use their own result types: discrete choice models return z -statistics and log-likelihood rather than t -statistics and residuals; panel models include within- R^2 and group structure; time series models include information criteria and forecast output. Unlike R's formula interface, the API uses explicit column names, because the LLM has already translated intent into specification.

The `EconError` type provides structured error information (singular matrix, convergence failure, insufficient clusters) that enables the LLM to offer actionable recovery guidance.

Validation uses automated test suites that compare Rust output against reference R packages on shared datasets. Coefficients, standard errors, and test statistics are verified to tolerances between 10^{-4} and 10^{-8} depending on the method. When intentional differences exist—for example, defaulting to HC1 rather than HC3 for robust standard errors, matching Stata's convention—these are documented and tested separately. The validation approach is described further in Section 5; Appendix H lists method-specific caveats and known differences.

3.3. Implemented methods

The `p2a-core` library implements 271 methods, exposed as individual MCP tools. Table 2 summarizes coverage, using standard abbreviations: heteroskedasticity and autocorrelation consistent (HAC), generalized least squares (GLS), high-dimensional fixed effects (HDFE), two-stage least squares (2SLS). Appendix H provides full documentation.

3.4. The MCP tool layer

The MCP server exposes tools organized into categories that mirror the core library (Table 3). Each tool has a JSON Schema defining required and optional parameters. Schema design minimizes required parameters to reduce LLM errors and provides sensible defaults (e.g., HC1 for robust standard errors). Tool descriptions are written to be interpretable by LLMs, specifying when each tool is appropriate. For example:

```
{
  "name": "regression_ols",
  "description": "Run OLS regression with optional robust SEs",
  "inputSchema": {
    "required": ["dataset", "y_var", "x_vars"],
    "properties": {
      "dataset": {"type": "string"},
      "y_var": {"type": "string"},
      "x_vars": {"type": "array", "items": {"type": "string"}},
      "robust": {"type": "string", "enum": ["hc0", "hc1", "hc2", "hc3"]},
      "cluster_var": {"type": "string"}
    }
  }
}
```

Note the distinction between MCP tools and core methods: tools serve as entry points that

Table 2: Implemented method categories

Category	Methods
Regression	OLS with HC0–HC3, clustered SEs, HAC, bootstrap, GLS, NLS, LOESS, splines, stepwise, quantile regression (Koenker and Jr. 1978)
Panel data	FE, RE, Hausman, HDFE, panel GLS, Arellano-Bond GMM (Arellano and Bond 1991), panel unit root tests (LLC, IPS, Fisher)
IV	2SLS, first-stage diagnostics, Sargan test, GMM (Hansen 1982)
Causal (design)	DiD, staggered DiD (Callaway and Sant’Anna 2021), Bacon decomposition (Goodman-Bacon 2021), ETWFE (Wooldridge 2021), sharp/fuzzy RD with CCT (Calonico, Cattaneo, and Titiunik 2014), multi-cutoff RD, synthetic control (Abadie, Diamond, and Hainmueller 2010), gsynth (Xu 2017), SCPI (Cattaneo, Feng, and Titiunik 2021)
Causal (treatment)	IPW, AIPW, matching (Ho, Imai, King, and Stuart 2011), CBPS (Imai and Ratkovic 2014), entropy balancing, GBM weighting, TMLE (van der Laan and Rose 2011), CTMLE, LTMLE, DML (Chernozhukov, Chetverikov, Demirer, Duflo, Hansen, Newey, and Robins 2018)
Discrete choice	Logit, probit, multinomial, conditional, mixed, ordered, negative binomial, ZIP/ZINB, hurdle
Time series	ARIMA, VAR, VARMA, VECM, IRF, Granger causality, STL/MSTL, Holt-Winters, structural TS, Kalman filter, GARCH, changepoint
Spatial	SAR, SEM, SAC (Anselin 1988), spatial probit, spatial panel, spatial GMM
Survival	Kaplan-Meier, log-rank, Cox PH, AFT, competing risks
Mediation	Causal mediation, natural effects, sensitivity (Cinelli and Hazlett 2020), E-values, Balke-Pearl bounds
Statistical tests	50+ tests: t -tests, ANOVA, MANOVA, χ^2 , Fisher, Wilcoxon, Kruskal-Wallis, Friedman, Shapiro-Wilk, KS, Tukey HSD

Note: Appendix H provides full documentation. Table A.1 lists unimplemented categories with recommended alternatives.

Table 3: MCP tool categories

Category	Examples	Tools
Data management	<code>load_dataset</code> , <code>describe_dataset</code>	7
Data transformation	<code>munge_filter</code> , <code>munge_join</code> , <code>munge_pivot</code>	19
Regression	<code>regression_ols</code> , <code>regression_diagnostics</code>	10
Panel data	<code>panel_fixed_effects</code> , <code>hausman_test</code>	5
Causal inference	<code>iv_2sls</code> , <code>diff_in_diff</code>	8
Discrete choice	<code>logit</code> , <code>probit</code>	2
Time series	<code>ts_arima_fit</code> , <code>ts_var</code>	6
Statistics	various hypothesis tests	30+
Visualization	<code>viz_scatter</code> , <code>viz_histogram_interactive</code>	7

Note: Tool count per category. Each tool corresponds to one or more **p2a-core** methods and is described by a JSON Schema that LLMs use for parameter extraction.

may invoke multiple methods. The `regression_ols` tool provides access to OLS with several covariance estimators (HC0–HC3, clustered, HAC), each a distinct method in **p2a-core**. This many-to-one mapping keeps the LLM’s selection task manageable while preserving access to the full library.

The full set of tool schemas consumes approximately 32,000 tokens, a substantial share of current context windows. For the built-in chat interface, the MCP server therefore exposes a curated subset of roughly 70 tools covering the most common analytical workflows; specialized tools are available on request or through the CLI. External LLM clients that connect via MCP receive the complete schema and can apply their own filtering.

Beyond executing methods, the system generates data-driven warnings when results suggest potential violations of identifying assumptions. Warnings are non-blocking—results are always returned—but they highlight conditions that warrant careful interpretation.

For instrumental variables, first-stage F -statistics are checked against the Stock–Yogo threshold of 10 (Stock and Yogo 2005). For staggered difference-in-differences, pre-trend tests are computed automatically. For matching and IPW, balance diagnostics flag standardized mean differences exceeding 0.1 (Austin 2011). For regression discontinuity, effective sample sizes within the bandwidth are checked against minimum thresholds.

Each warning includes a severity level, a diagnostic message, the assumption at risk, and suggested remediation steps. This structured information enables the LLM to provide recovery guidance rather than generic failure messages. The thresholds are deliberately conservative (e.g., $F < 10$ for weak instruments, $\text{SMD} > 0.1$ for balance), which may produce false positives in settings where these rules of thumb are known to be overly strict. We prefer occasional unnecessary warnings to missed violations; users can evaluate the diagnostic evidence and disregard warnings they judge inapplicable.

3.5. Implementation choices

The choice of Rust follows from the principles. Principle 2 (pre-validation) requires an implementation language where validated methods remain correct across deployments—Rust’s compile-time memory safety guarantees eliminate buffer overflows, use-after-free, and data races that can cause incorrect results in numerical software. Principle 3 (data locality) benefits from compiled performance: the same analytics engine runs efficiently on the user’s machine, not just on cloud servers. A single codebase compiles to standalone binaries, WebAssembly for browsers, and shared libraries for language integration.

The ecosystem provides the necessary components: **polars** for data manipulation (Polars Contributors 2024), **ndarray** for array operations, and **faer** for linear algebra. All computations use 64-bit floating-point arithmetic throughout, matching R’s precision. To date, no econometrics or data analytics library has been available for Rust; **p2a-core** addresses this gap.

Several additional design choices merit brief explanation. For conversation persistence, the web application uses SurrealDB, an embedded document database for storing conversation history. Its embedded operation requires no external service; its flexible schema accommodates evolving conversation structures without migrations. Analytical datasets are held in memory during sessions and not persisted, limiting storage to lightweight metadata.

When users request methods outside the implemented set, the LLM infers scope from the

tool schema. In some cases, it maps to the closest available alternative and explains the substitution; in others it reports unavailability.

Regarding security, tools have fixed descriptions embedded in the server binary—external modification requires rebuilding. No arbitrary code execution is possible. Tool calls are logged and can be inspected. Dataset access requires explicit loading; the server has no ambient filesystem access. Nevertheless, users should avoid loading untrusted data files that could contain injection payloads, and prefer local LLM deployment for high-security environments.

4. Examples

This section demonstrates **prompt2analytics** through two interfaces: the natural language chat interface powered by large language models, and the command-line interface for scripted workflows. Both interfaces access the same underlying Rust analytics engine.

We use the Grunfeld investment data (Baltagi 2013), a classic panel dataset with 200 observations (10 firms over 20 years) containing firm investment, market value, and capital stock. The dataset is included in the **plm** package; we provide it as `grunfeld.csv` in the supplementary materials.

The early examples show the system working as intended; later examples (Section 4.3) demonstrate failure modes—incorrect tool selection, missing parameters, and ambiguous prompts—to set realistic expectations before the formal evaluation in Section 5. Appendix I provides additional examples including time series analysis, data cleaning workflows, and a comparison with equivalent R code.

4.1. Chat interface

The chat interface enables natural language interaction with the analytics engine. Users describe their analysis goals in plain language, and the LLM translates these requests into structured tool calls executed by the MCP server. This interface is useful for exploratory data analysis and for users less familiar with programming syntax. The examples in this section were generated using Claude 3.5 Sonnet via the MCP integration; other models (GPT-4o-mini, Llama 3.3 70B, etc.) produce similar results with minor variations in explanation style. A typical session begins by loading data and examining its structure:

User: Load the Grunfeld investment dataset from grunfeld.csv and show me what's in it.

I'll load the dataset and describe its contents.

```
**Dataset loaded: grunfeld**
- Rows: 200
- Columns: 5
```

Column	Type	Description
firm	Integer	Firm identifier (1-10)

year	Integer	Year (1935-1954)	
inv	Float64	Gross investment	
value	Float64	Market value of firm	
capital	Float64	Stock of plant and equipment	

This is a balanced panel dataset with 10 firms observed over 20 years, commonly used to demonstrate panel data methods.

The next example shows cross-sectional regression with heteroskedasticity-consistent standard errors.²

*User: Run a regression of investment on market value and capital stock.
Use robust standard errors.*

Running OLS regression with HC1 robust standard errors...

****OLS Regression Results****

Dependent variable: inv

Variable	Coef	Std Err	t	P> t	
-----	-----	-----	-----	-----	
(Intercept)	-42.714	11.575	-3.690	0.000	
value	0.116	0.007	16.967	0.000	
capital	0.231	0.049	4.721	0.000	

R-squared: 0.812

Adj. R-squared: 0.810

Observations: 200

Standard errors: HC1 (heteroskedasticity-robust)

Both market value and capital stock are highly significant predictors of investment. A \$1 million increase in market value is associated with approximately \$116,000 more investment, holding capital constant.

For panel data, users can request fixed effects estimation in natural language:

*User: The data has a panel structure. Run a fixed effects regression
controlling for firm-specific effects.*

Running within-transformation fixed effects estimation...

****Fixed Effects (Within) Estimation****

Dependent variable: inv

Entity fixed effects: firm (10 levels)

²HC1 is the default robust standard error estimator when robust SEs are requested. HC1 applies a degrees-of-freedom correction ($n/(n - k)$) to HC0, improving small-sample performance.

Variable	Coef	Std Err	t	P> t
value	0.110	0.012	9.288	0.000
capital	0.310	0.017	17.867	0.000

R-squared (within): 0.767

Observations: 200

Groups: 10

F-statistic: 309.01 (p-value: 0.000)

After controlling for time-invariant firm characteristics, the coefficient on capital increases from 0.23 to 0.31, suggesting that pooled OLS underestimated the association between capital stock and investment. (Note: This change in coefficient reflects removal of omitted variable bias, not necessarily identification of a causal effect.)

Users can request specification tests without knowing the technical details:

User: Should I use fixed effects or random effects for this analysis?

I'll run a Hausman test to help determine the appropriate specification.

****Hausman Test Results****

H0: Random Effects is consistent and efficient

H1: Fixed Effects is consistent; Random Effects is inconsistent

Chi-squared statistic: 0.0003

Degrees of freedom: 2

P-value: 0.9999

****Recommendation**:** Random Effects

The test fails to reject the null hypothesis. The near-zero chi-squared statistic indicates virtually no systematic difference between the FE and RE coefficient estimates for this dataset.

****Note**:** Extreme p-values like 0.9999 warrant interpretation with caution. Such values can arise from small test statistics (as here), but also from numerical issues or model misspecification. The Hausman test assumes both estimators are consistent under the null, which requires correct functional form, homoskedastic errors, and no omitted variables correlated with the regressors. The test statistic may be unreliable with small samples, heteroskedasticity, or serial correlation. Users should treat very high p-values as suggestive rather than definitive.

In this example, the test statistics (chi-squared, p-value, degrees of freedom) and the recommendation come from the Rust backend; the interpretive caveat about extreme p-values is gen-

erated by the LLM. This division is typical: the computation layer returns structured results and simple diagnostics, and the language model adds contextual interpretation—including caveats about assumptions and limitations that depend on the specific results observed.

For datasets with multiple categorical dimensions, users can request multi-way fixed effects:

User: Control for both firm and year fixed effects in the regression.

Running two-way fixed effects using alternating projections...

****High-Dimensional Fixed Effects Estimation****

Dependent variable: inv

Fixed effects: firm (10 levels), year (20 levels)

Variable	Coef	Std Err	t	P> t
-----	-----:	-----:	-----:	-----:
value	0.118	0.019	6.137	0.000
capital	0.358	0.054	6.579	0.000

R-squared (within): 0.720

Observations: 200

F-statistic: 217.44 (p-value: 0.000)

Users can also request diagnostic tests in natural language:

User: Check the regression assumptions for the OLS model.

Running regression diagnostics...

****Jarque-Bera Test (Normality of residuals)****

Statistic: 89.57

P-value: 0.000

Conclusion: Residuals deviate significantly from normality.

****Breusch-Pagan Test (Heteroskedasticity)****

Statistic: 55.25

P-value: 0.000

Conclusion: Evidence of heteroskedasticity; robust SEs recommended.

****Durbin-Watson Statistic (Autocorrelation)****

Statistic: 0.358

Interpretation: Positive autocorrelation detected (expected in panel data).

****Variance Inflation Factors****

value: 1.31

capital: 1.31

No multicollinearity detected (VIF < 5).

4.2. Command-line interface

The command-line interface provides programmatic access for scripted workflows. Commands follow a hierarchical structure with sessions maintaining state:

```
# Load data and run OLS with robust standard errors
p2a --session analysis.json data load grunfeld.csv --name grunfeld
p2a --session analysis.json reg ols grunfeld \
    -y inv -x value capital --robust hc1

# Panel fixed effects
p2a --session analysis.json panel fe grunfeld \
    -y inv -x value capital --entity firm

# Export session to reproducible script
p2a script export analysis.json -o reproduce.sh
```

Note: Backslash line continuation (\) works in Bash, Zsh, and most POSIX-compatible shells. Windows Command Prompt users should write commands on a single line or use PowerShell's backtick (`) for continuation.

All commands support JSON output (-F json) for integration with other tools and scripts:

```
p2a -F json reg ols grunfeld -y inv -x value capital --robust hc1

{
  "coefficients": {
    "(Intercept)": -42.714, "value": 0.116, "capital": 0.231
  },
  "std_errors": {
    "(Intercept)": 11.575, "value": 0.007, "capital": 0.049
  },
  "t_values": {
    "(Intercept)": -3.690, "value": 16.967, "capital": 4.721
  },
  "p_values": {
    "(Intercept)": 0.000, "value": 0.000, "capital": 0.000
  },
  "r_squared": 0.812, "adj_r_squared": 0.810,
  "n_obs": 200, "cov_type": "HC1"
}
```

This enables integration with analysis pipelines, for example, piping results to jq for extraction or to Python/R scripts for further processing. The session mechanism records all commands and can export them as an executable shell script for exact reproduction.

4.3. Realistic use and error handling

The preceding examples show successful interactions. Real use involves more iteration. This section shows what failures look like, how to recover, and what accuracy to expect (Section 5 provides detailed numbers).

Vague requests prompt the system to ask for clarification rather than guess:

User: Run a regression with firm effects.

I can estimate this model, but need clarification:

1. ****Dependent variable****: Which column should be the outcome?
2. ****Independent variables****: Which predictors should be included?
3. ****Fixed vs Random effects****: Should firm effects be treated as fixed (within estimator) or random (GLS)?

Please specify, e.g., "Regress inv on value and capital with firm fixed effects"

The following workflow illustrates common failure modes: wrong method selected, parameters extracted incorrectly, user corrections, and eventual success.

*User: Analyze whether the training program affected employee performance.
The data has employees observed before and after training, with a control group that didn't receive training.*

I'll run a t-test comparing performance between training and control groups.

```
**Two-sample t-test**
Training group mean: 3.82
Control group mean: 3.45
Difference: 0.37 (95% CI: [0.21, 0.53])
p-value: 0.000
```

The training group shows significantly higher performance.

*User: That's not right - I need a difference-in-differences analysis,
not just a comparison of means. I have before/after data.*

You're right, I apologize. Let me run a difference-in-differences estimation to properly account for the before/after structure.

```
**Difference-in-Differences Estimation**
Treatment indicator: training_group
Time indicator: post_training
```

Variable	Coef	Std Err	t	P> t
post_training	0.12	0.08	1.50	0.134

training_group		0.31		0.11		2.82		0.005	
post × training (ATT)		0.24		0.09		2.67		0.008	

The ATT estimate is 0.24 performance points.

User: Good, but I asked for clustered standard errors at the employee level to account for repeated observations.

Let me re-run with clustered standard errors:

****Difference-in-Differences**** (Clustered by employee, 847 clusters)

Variable		Coef		Cluster SE		t		P> t	
-----		-----		-----		-----		-----	
post_training		0.12		0.11		1.09		0.276	
training_group		0.31		0.14		2.21		0.027	
post × training (ATT)		0.24		0.12		2.00		0.046	

With clustered SEs, the ATT remains significant at the 5% level, though standard errors are larger than the unclustered estimates.

User: Now check for pre-trends to validate the parallel trends assumption.

****Pre-trend Test****

Testing whether treatment and control groups had parallel trends in the pre-treatment period...

Pre-period		Diff		Std Err		t		P> t	
-----		-----		-----		-----		-----	
t-3		-0.02		0.09		-0.22		0.826	
t-2		0.05		0.08		0.63		0.531	
t-1		0.01		0.08		0.13		0.900	

Joint test (H0: all pre-period diffs = 0):

F-statistic: 0.18, p-value: 0.912

No evidence of differential pre-trends; parallel trends assumption is supported (though not proven) by this test.

This workflow required three correction turns: (1) correcting method selection from a t-test to DiD; (2) adding clustered standard errors; and (3) requesting pre-trend diagnostics. The final result is correct.

Practical tips from using the system: naming variables directly (“regress inv on value”) is more reliable than using pronouns (“use the same variables”), and users should always verify which parameters the model extracted before trusting the output.

The system also handles common failure modes with explanatory guidance.

The system detects multicollinearity and reports it with suggested actions:

User: Add years_since_hire to the model alongside tenure.

****Error: Perfect multicollinearity detected****

The design matrix is rank-deficient. Column `years\_since\_hire` is a linear combination of existing columns (correlation with tenure: 1.000).

****Suggested actions:****

1. Remove one of the collinear variables
2. If both are theoretically important, consider which better captures your construct of interest

Similarly, convergence failures are caught and explained:

User: Fit a logit model with department fixed effects.

****Warning: Convergence issues detected****

****Issue:**** Perfect prediction in some groups

- Department "Legal" has 0% bonus rate (23 observations)

****Options:****

1. Drop singleton departments
2. Use a linear probability model (robust to separation)
3. Use penalized likelihood (Firth correction)

Running linear probability model with department FE instead...

The system detects problems before estimation fails, explains statistical issues rather than computational symptoms, and provides actionable suggestions. When requests exceed capabilities, it clearly states limitations and points to external tools.

These examples illustrate what the system can do; the next section measures how reliably it does so. We evaluate tool selection accuracy across 12 models and numerical correctness of the underlying Rust implementations.

5. How well does it work?

Two kinds of accuracy matter: does the LLM select the right tool with the right parameters? And does the Rust implementation produce correct numerical results? This section evaluates both.

5.1. Tool selection evaluation

We evaluated LLM performance on 87 test cases spanning regression, panel data, causal inference, discrete choice, time series, hypothesis testing, machine learning, and visualization. Each case presents a natural language prompt; the model must select the appropriate tool

from the **prompt2analytics** schema. Responses are scored as exact (correct tool), acceptable (valid alternative), category (correct domain, wrong method), or failed. Accuracy is computed as $(\text{exact} + \text{acceptable}) / \text{total}$. All 87 cases were scored by the first author; the scoring rubric and individual scores are provided in the supplementary materials (`test_cases.json`), enabling independent verification. The definition of “acceptable” was intentionally narrow: only tools that perform a statistically equivalent analysis qualify (e.g., `regression_ols` with HC1 for a request mentioning robust standard errors). Alternative approaches that change the estimator (e.g., GLS instead of OLS) are scored as “category.”

All evaluations used temperature = 0 for deterministic outputs. The system prompt instructs the model to select a single tool from the provided MCP schema and return a structured JSON response with tool name and parameters. The complete system prompt is provided in the supplementary materials file `llm_eval_prompts.txt`; the summary form is:

“You are an assistant that selects statistical tools. Given a user request, choose exactly one tool from the available schema and provide the required parameters. Respond with valid JSON containing `tool_name` and `parameters`.”

Table 4 lists the model versions used for evaluation. All models were accessed through cloud APIs (Anthropic, OpenAI, or OpenRouter). API-served models may change over time, so exact replication of cloud-hosted results may not be possible; results for proprietary models (Claude, GPT) should be treated as a snapshot. For open-source models (Qwen, Llama, Mistral, Gemma), exact reproduction is possible by serving fixed weights locally via Ollama. The supplementary materials include the evaluation harness, all 87 test cases as JSON, and the system prompt used for scoring, so that new models can be evaluated against the same benchmark.

Table 4: Models evaluated

Model	Version ID	Provider
Claude 3.5 Haiku	<code>claude-3-5-haiku-20241022</code>	Anthropic API
GPT-4o-mini	<code>gpt-4o-mini-2024-07-18</code>	OpenAI API
GPT-5 Nano ^a	<code>gpt-5-nano-2025-08-07</code>	OpenAI API
GPT-4.1 Nano	<code>gpt-4.1-nano-2025-04-14</code>	OpenAI API
Qwen 2.5 72B	<code>qwen-2.5-72b-instruct</code>	OpenRouter
Llama 3.3 70B	<code>llama-3.3-70b-instruct</code>	OpenRouter
Mistral Small 24B ^b	<code>mistral-small-24b-instruct-2501</code>	OpenRouter
DeepSeek V3 ^a	<code>deepseek-chat</code>	OpenRouter
Nemotron Nano 9B	<code>nemotron-nano-9b-v2</code>	OpenRouter
Qwen3 8B	<code>qwen3-8b</code>	OpenRouter
Qwen 2.5 7B ^b	<code>qwen-2.5-7b-instruct</code>	OpenRouter
Ministral 3B	<code>ministral-3b</code>	OpenRouter

Note: Accessed January 2026. Open-source models served through OpenRouter.

^a Single-turn evaluation only. ^b Extended evaluation only. Cloud-served model weights may be updated without notice; for reproducible local deployment, pin Ollama models by digest (e.g., `ollama pull qwen2.5:72b@digest`).

Figure 3 presents overall accuracy by model. The results demonstrate that tool selection—unlike code generation—is achievable even by smaller models. Claude 3.5 Haiku and Qwen 2.5 72B (an open-source model) achieve 100% accuracy. Notably, Ministral 3B, a model

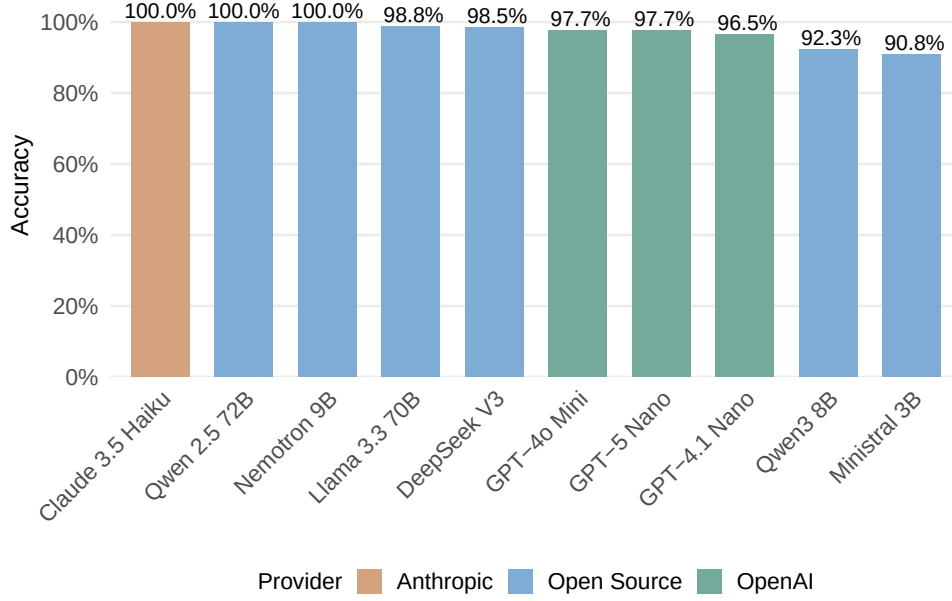


Figure 3: Tool selection accuracy across models (87 tests). Accuracy = (exact + acceptable) / total. Even Ministral 3B (3B parameters) exceeds 90%.

small enough to run on consumer hardware with only 6 GB VRAM, achieves 90.8% accuracy, making it viable for local deployment scenarios where hardware constraints preclude larger models.

Figure 4 shows accuracy by category. Performance is uniformly high across most categories, with errors concentrated in edge cases requiring fine distinctions between related methods. For instance, Ministral 3B shows lower accuracy on time series (66.7%) where distinguishing between VAR, VARMA, and VECM specifications proves challenging for the smaller model. Specific confusion patterns include: selecting VAR when VECM is appropriate (failing to recognize cointegration requirements), choosing ARIMA for multivariate series that require VAR, and defaulting to simpler univariate methods when the prompt describes multiple interrelated series.

Tables B.1 and B.2 in Appendix B provide detailed breakdowns of model performance and per-category accuracy. Appendix C reports API response latencies, though these figures reflect cloud-hosted inference and do not directly apply to local deployment scenarios.

5.2. Failure modes

Understanding system behavior under failure conditions is essential for informed use. Three categories of failure merit discussion. First, when the LLM selects an inappropriate statistical method, the system proceeds with the selected method and returns results. Users must recognize the mismatch between request and execution. Common patterns include ambiguous specifications (“regression with firm effects” could invoke OLS, fixed effects, or random effects), similar method confusion (VAR/VECM/VARMA), and capability boundary cases where requests for unimplemented methods map to superficially similar available tools. Users

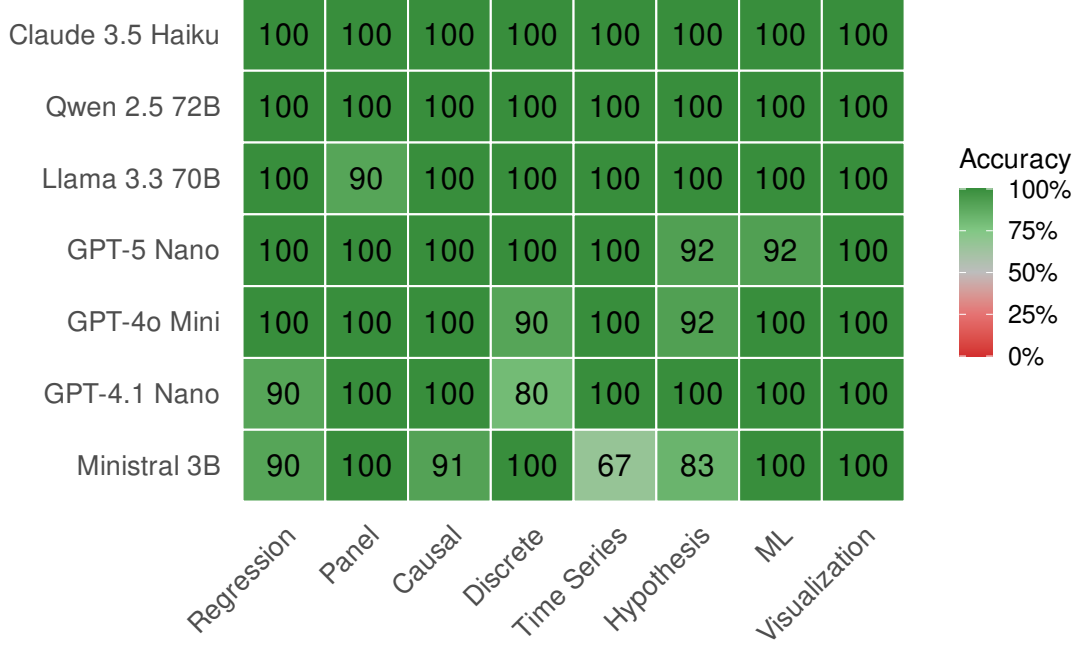


Figure 4: Accuracy by test category. Darker cells indicate lower accuracy. Errors concentrate in ambiguous method selection.

can mitigate misselection by specifying methods explicitly rather than describing goals abstractly.

Table 5 summarizes the most frequent misselection patterns from the naturalistic prompt evaluation (891 test instances across 9 models). Four categories emerge. *Related test substitution* (e.g., ADF $\rightarrow$ Phillips-Perron) selects a valid alternative from the same test family—often harmless, but not what was requested. *Specific $\rightarrow$ omnibus* maps a named diagnostic (Breusch-Pagan, VIF) to the general diagnostics bundle, losing precision. *Method-level confusion* produces a fundamentally different estimator (NLS $\rightarrow$ OLS, hierarchical clustering $\rightarrow$ DBSCAN); the “clustered SEs $\rightarrow$ K-means” case illustrates how lexical overlap in the word “cluster” misleads smaller models. *Scope confusion* selects a simpler variant that omits a requested feature (event study $\rightarrow$ basic DiD, two-way FE $\rightarrow$ one-way FE).

Second, even when the correct method is selected, the LLM may extract incorrect parameters. Manual inspection of recorded tool calls across all evaluated models shows that core parameters—dependent variable, independent variables, entity and time identifiers—are almost always correct when the tool itself is correct. The main source of imprecision is omission of optional parameters: a model may select `regression_ols` with the right `y_col` and `x_cols` but omit the `robust` option, falling back on the tool’s default (HC1). Because the schema defines sensible defaults for optional parameters, these omissions typically produce a valid analysis that differs only in secondary specification choices. Explicit phrasing (“with HC1 robust standard errors”) yields more reliable extraction than implicit expectations. We do not report a formal parameter extraction metric because our evaluation showed that scoring precision and recall against expected parameter names is confounded by name aliasing across

Table 5: Most frequent tool misselection patterns (naturalistic prompts)

Expected	Selected	<i>n</i>	Category
ADF test	Phillips-Perron	17	Related test
Breusch-Pagan	Diagnostics bundle	16	Specific → omnibus
Auto-ARIMA	AR	15	Method confusion
Durbin-Watson	Box-Ljung	10	Related test
Two-way FE	One-way FE	9	Scope confusion
Hierarchical	DBSCAN	9	Method confusion
NLS	OLS	8	Method confusion
Welch <i>t</i>	Standard <i>t</i>	8	Related test
VIF	Diagnostics bundle	8	Specific → omnibus
Histogram	Describe dataset	8	Scope confusion

Note: Counts aggregated across 9 models on 99 naturalistic prompts (891 model-prompt pairs). Naming aliases (e.g., `panel_fe` vs. `panel_fixed_effects`) excluded. Full scored results are in the supplementary materials.

tool schemas (e.g., `cluster_col` vs. `cluster_var`), making the metric unreliable as a measure of actual analysis correctness.

These observations suggest that for curated prompts, the dominant failure mode is tool misselection (5–10% of requests), not parameter error. For naturalistic prompts, where tool accuracy drops to 58–70%, both misselection and ambiguous parameter specification contribute. In all cases, users can inspect the tool call before execution, correct misselected methods or missing parameters, and iterate.

Third, categorical (string) variables are automatically converted to dummy variables using treatment coding (one level omitted as reference). The system detects string columns, creates indicator variables, reports the reference category, and warns when categorical variables have many levels (> 50).

5.3. Numerical validation against R

The preceding subsections evaluated the LLM layer. This subsection evaluates the computation layer: does the Rust implementation produce the same numbers as established R packages?

We validate all methods against reference implementations using sample-size-dependent tolerances (looser for small samples where ill-conditioning amplifies differences, tighter for large samples). Reference packages include `lm()`/`sandwich` for OLS, `plm/lfe` for panel data, `glm()` for discrete choice, and `forecast` for time series. All validation tests use fixed random seeds (seed = 42) for reproducibility; reference R scripts are provided in the supplementary materials.

The Longley dataset (Longley 1967), with its extreme multicollinearity (condition number $> 10^9$), provides a stringent test. Table 6 shows coefficients match R’s `lm()` to at least 8 significant digits.

Heteroskedasticity-consistent standard errors (HC0–HC3) are validated against `sandwich` (Zeileis 2004). HC0 and HC1 match to $< 10^{-8}$; HC2 and HC3 match to $< 10^{-7}$, reflecting algorithm-

Table 6: OLS validation: Longley dataset

Coefficient	R <code>lm()</code>	prompt2analytics	Difference
(Intercept)	-3482.259	-3482.259	$< 10^{-8}$
GNP.deflator	0.01506	0.01506	$< 10^{-8}$
GNP	-0.03582	-0.03582	$< 10^{-8}$
Unemployed	-0.02020	-0.02020	$< 10^{-8}$
Armed.Forces	-0.01033	-0.01033	$< 10^{-8}$
Population	-0.05110	-0.05110	$< 10^{-8}$
Year	1.82915	1.82915	$< 10^{-8}$
R^2	0.9955	0.9955	$< 10^{-4}$

Note: $n = 16$, $k = 6$. The Longley dataset’s extreme multicollinearity (condition number $> 10^9$) provides a stringent near-singular test case: the design matrix is close to rank-deficient, exercising the Cholesky-based solver’s tolerance framework. All coefficients match to at least 8 significant digits, confirming that the implementation handles ill-conditioned problems correctly.

mic differences in leverage computation (QR decomposition vs. direct matrix multiplication). Both approaches are numerically valid; the discrepancy is floating-point accumulation, not methodology.

Fixed effects coefficients on the Grunfeld dataset (10 firms, 20 years, $n = 200$) match **plm** (Croissant and Millo 2008) to $< 10^{-6}$; the within R^2 matches to four decimal places. The Hausman test statistic matches to $< 10^{-4}$. Two-way HD FE via the MAP algorithm (Gaure 2013) matches **lfe**’s `fe1m()` to $< 10^{-5}$.

Beyond the methods shown above, the validation suite covers over 200 methods with more than 1,500 individual test cases (334 specifically targeting validation against R reference implementations). Statistical tests (t -tests, ANOVA, chi-squared, Fisher’s exact, Wilcoxon, Shapiro-Wilk, Kolmogorov-Smirnov) are validated against R’s **stats** package. Discrete choice models (logit, probit) match `glm()` coefficients and standard errors. Time series methods (VAR, ARIMA) match **vars** and **forecast**. The **p2a-core** test suite comprises 1,962 test functions. Continuous integration (GitHub Actions) runs the full test suite, **clippy** linting, and format checks on every push to **main** and on all pull requests. Line-level code coverage is not currently tracked as a summary metric; instead, we prioritize validation breadth—each implemented method has at least one cross-validated test against an R reference. The CI pipeline does *not* execute the R reference scripts; cross-language validation is run manually before releases and the reference outputs are checked into the repository. All Rust-side validation tests can be run via `cargo test -p p2a-core`.

For complex causal inference and treatment effect methods, validation depth varies. Canonical 2×2 difference-in-differences, IV/2SLS, IPW, doubly robust estimation (AIPW), and causal mediation analysis are cross-validated against R packages—`lm()` for DiD, **AER** for IV, manual Horvitz–Thompson/Hájek estimators for IPW, and the **mediation** package—on synthetic

datasets with known treatment effects (e.g., true $ATT = 5$ for DiD, true $\beta = 1$ for IV, true $ATE = 0.75$ for IPW/AIPW). Propensity score matching is validated against **MatchIt** (Ho *et al.* 2011) for balance diagnostics (standardized mean differences, variance ratios). Regression discontinuity is validated against the **rdrobust** framework (Calonico *et al.* 2014), checking that local linear estimates recover the true discontinuity within tolerance. FEGLM is validated against **fixest**’s `feglm()` (Bergé 2018) for coefficient sign, pseudo R^2 , and the logit/probit scaling ratio (≈ 1.6). TMLE has eight validation tests that verify the targeting step, clever covariate formula, influence curve properties, propensity score truncation, and counterfactual predictions on simulated data; these check mathematical identities and DGP-implied ranges rather than exact R output values. In total, 55 R reference scripts (one per validated method) and the Rust benchmarking runner are provided in the supplementary materials.

Several complex methods—staggered DiD (Callaway and Sant’Anna 2021), extended TWFE, Goodman-Bacon decomposition, Double/Debiased ML, synthetic control, SCPI, CTMLE, LTMLE, CBPS, and entropy balancing—have internal consistency tests (correct output dimensions, convergence on simulated DGPs, coefficient sign verification, weight-sum constraints) but lack direct numerical cross-validation against R reference packages. The corresponding R implementations (e.g., **did**, **DoubleML**, **Synth**, **CBPS**) use different default tuning parameters, cross-validation splits, and optimization algorithms that complicate exact numerical comparison; matching output requires careful configuration alignment that is not yet in place. Extending cross-validation coverage to these methods is a priority for future releases.

Appendix G documents tolerance thresholds by method category and sample size. Appendix H lists known differences from reference implementations (default robust SE type, convergence tolerances, R^2 variant selection)—none affecting the validity of statistical inference.

For core panel methods, **prompt2analytics** provides comparable coverage to specialized R packages; the primary remaining gap is Conley spatial standard errors. The distinguishing features are the natural language interface and fully local execution without R dependencies. Users requiring maximum performance for high-dimensional fixed effects should note that **fixest** (Bergé 2018) likely outperforms **prompt2analytics** due to its highly optimized C++ implementation. **prompt2analytics** complements rather than replaces specialized tools. Having established the system’s accuracy and feature coverage, the next section examines whether it can run efficiently—and entirely locally—on consumer hardware.

6. Performance and deployment

The three principles require software fast enough to run locally and models small enough to fit on consumer hardware. This section presents performance benchmarks against R and describes fully local deployment configurations where all components—including the language model—run on the user’s machine.

6.1. Performance comparison with R

Table 7 presents benchmark results across 41 method configurations at $n = 100,000$ observations with multiple iterations per method—100 for fast methods, 50 for moderate, 20 for

slow—after 10 warmup iterations (median timing reported; see Appendix G for details). Both R and Rust timings measure computation only, excluding data loading and process startup, ensuring a fair comparison of algorithmic performance. Across 42 method configurations, the median speedup is $4.1\times$, with wide variation by method category reflecting the relative weight of interpreter overhead versus numerical computation.

The majority of methods show speedups of $2\text{--}10\times$: OLS ($6.9\times$), IV/2SLS ($5.2\times$), Hausman ($4.7\times$), MSTL decomposition ($4.4\times$), Kaplan-Meier ($4.1\times$), and t-tests ($3\text{--}7\times$). Compute-intensive iterative methods see larger gains: quantile regression ($44\times$), Wilcoxon rank-sum ($23\times$), OLS with robust standard errors ($15\text{--}22\times$ for HC0–HC3), panel fixed effects ($21\times$), and DBSCAN ($14.5\times$). One extreme outlier—Granger causality at $660\times$ —likely reflects implementation differences in R’s `lmtest::grangertest` (which fits full VAR models per lag) rather than a genuine language advantage, and should be interpreted with caution.

Methods near parity include two-way ANOVA, AR, and negative binomial regression (all $\sim 1\times$). R outperforms Rust for PCA ($0.03\times$), NLS ($0.1\times$), diagnostics ($0.2\times$), and ACF ($0.4\times$)—operations where R’s highly optimized LAPACK bindings or vectorized C implementations dominate. For high-dimensional fixed effects, **fixest** (Bergé 2018) provides a highly optimized C++ implementation that likely outperforms both **plm** and **prompt2analytics** at large scale; our HDFFE benchmark against **plm** shows only $1.3\times$ speedup. At smaller sample sizes ($n < 1,000$), speedups can be much larger—for example, panel fixed effects at $n = 100$ is $184\times$ faster—but these figures primarily reflect R’s interpreter overhead (formula parsing, factor handling) rather than computational advantages. Complete benchmark results are in Appendix G; reproduction instructions are in Appendix F.

The performance advantages stem from Rust’s ownership system (stack allocation and in-place mutation without garbage collection), the **faer** library’s SIMD-optimized linear algebra, the column-based API that eliminates formula parsing, and the single-language design that avoids data serialization between components. **prompt2analytics** performs best on computationally intensive iterative methods (panel fixed effects, robust standard errors, quantile regression, DBSCAN) where compile-time optimizations and memory efficiency compound across iterations. For single-pass operations dominated by optimized linear algebra (PCA, ACF, NLS), R’s BLAS bindings provide comparable or superior performance.

For users with CUDA-capable hardware, **p2a-core** provides optional GPU acceleration via cuBLAS and cuSOLVER. The `cuda` feature flag enables GPU code paths at compile time; at runtime, a singleton context attempts CUDA initialization and falls back silently to CPU if no device is available. Public function signatures are unchanged—users do not modify their code.

Not all operations benefit from GPU offloading. Transfer overhead between host and device memory dominates for small problems, and bandwidth-bound operations (e.g., matrix-vector products) can be faster on CPU even for large n . The dispatch logic therefore checks two conditions before routing to GPU: a minimum problem dimension (e.g., $k \geq 30$ columns for $\mathbf{X}'\mathbf{X}$) and a minimum computational intensity (e.g., $nk^2 \geq 2 \times 10^7$). Both thresholds are configurable via environment variables, allowing users to tune dispatch for their specific hardware. A shape guard additionally prevents GPU dispatch for tall-skinny matrix multiplications, where the CPU path is substantially faster.

Table 8 summarizes the operations with GPU code paths and observed speedups on an NVIDIA DGX Spark (Grace Blackwell, 12 GB VRAM, unified memory). The largest gains

Table 7: Performance benchmark results at $n = 100,000$

Category	Method	n	R (ms)	Rust (ms)	Speedup
Causal	IV/2SLS	100,000	264.6	50.7	$5.2\times$
Discrete	Logit	100,000	406.9	157.4	$2.6\times$
Discrete	Probit	100,000	499.6	222.1	$2.2\times$
Discrete	NegBin	100,000	1932.6	1440.6	$1.3\times$
ML	DBSCAN	100,000	52331.9	3614.2	$14.5\times$
ML	Kmeans	100,000	782.7	349.5	$2.2\times$
ML	PCA	100,000	18.9	600.0	$0.03\times$
Munging	Group By	100,000	71.2	3.5	$20.6\times$
Munging	Sort	100,000	28.5	4.1	$6.9\times$
Munging	Standardize	100,000	10.1	1.5	$6.9\times$
Munging	Filter	100,000	14.5	9.6	$1.5\times$
Panel	Panel Fe	100,000	433.5	20.3	$21.3\times$
Panel	Hausman	100,000	1094.7	235.4	$4.7\times$
Panel	Panel Re	100,000	1048.2	230.8	$4.5\times$
Panel	HDFE	100,000	54.0	41.0	$1.3\times$
Regression	Quantreg	100,000	22087.5	501.4	$44.0\times$
Regression	Ols+Hc1	100,000	308.2	14.0	$22.0\times$
Regression	Ols+Hc2	100,000	308.0	16.7	$18.4\times$
Regression	Ols+Hc3	100,000	306.2	19.0	$16.1\times$
Regression	Ols+Hc0	100,000	289.8	18.8	$15.4\times$
Regression	Ols	100,000	57.7	8.3	$6.9\times$
Regression	Ols+Clustered	100,000	98.9	72.6	$1.4\times$
Regression	Diagnostics	100,000	244.1	1475.1	$0.2\times$
Regression	NLS	100,000	73.4	596.1	$0.1\times$
Statistics	Wilcoxon	100,000	690.5	29.9	$23.1\times$
Statistics	Kruskal Wallis	100,000	506.6	49.2	$10.3\times$
Statistics	Ttest One Sample	100,000	3.5	0.5	$7.2\times$
Statistics	Ttest Two Sample	100,000	6.3	2.1	$3.0\times$
Statistics	Shapiro Wilk	100,000	1.5	0.5	$3.0\times$
Statistics	Anova Oneway	100,000	124.0	45.0	$2.8\times$
Statistics	Anova Twoway	100,000	81.4	78.7	$1.0\times$
Statistics	Chi-sq Gof	100,000	7.3	18.8	$0.4\times$
Survival	Kaplan Meier	100,000	441.5	107.1	$4.1\times$
Survival	Cox Ph	100,000	629.1	274.1	$2.3\times$
Time Series	Granger	100,000	5675.0	8.6	$659.9\times$
Time Series	MSTL	100,000	213.9	48.2	$4.4\times$
Time Series	STL	100,000	176.0	64.5	$2.7\times$
Time Series	Holt Winters	100,000	2939.8	1124.5	$2.6\times$
Time Series	Ar	100,000	26.2	27.0	$1.0\times$
Time Series	Changepoint	100,000	67.9	71.9	$0.9\times$
Time Series	ACF	100,000	29.9	71.0	$0.4\times$

Note: Median execution time in milliseconds. Speedup = R time / Rust time. Methods with speedup $> 1\times$ are faster in Rust; methods $< 1\times$ are faster in R, typically reflecting R's optimized LAPACK/C internals for those operations.

appear in $\mathbf{X}'\mathbf{X}$ computation (up to $7\times$ for $k \geq 100$) and PCA via the covariance path (up to $13\times$ for tall matrices). K-means distance computation and sandwich estimators for heteroskedasticity-robust standard errors see more modest improvements of $2\text{--}3\times$.

Table 8: GPU-accelerated operations and observed speedups

Operation	Use in	Speedup	Condition
$\mathbf{X}'\mathbf{X}$ (DGEMM)	OLS, panel FE/RE, IV, robust SEs	$2\text{--}7\times$	$k \geq 30$
PCA covariance	Dimensionality reduction	$4\text{--}13\times$	$n > 4k$
K-means distances	Clustering	$2\text{--}3\times$	$d \geq 20$
Sandwich $\mathbf{X}'\boldsymbol{\Omega}\mathbf{X}$	HC0–HC3 robust SEs	$2\text{--}3\times$	$k \geq 30$

Note: Measured on NVIDIA DGX Spark (Grace Blackwell, unified memory). Speedups are relative to the CPU path on the same machine (ARM Cortex-X925 with OpenBLAS). Results will differ on discrete GPUs where host–device transfer costs are higher.

Most econometric applications involve moderate numbers of regressors ($k < 30$), where CPU execution is faster. GPU acceleration is therefore not part of the default build; it targets high-performance computing scenarios with many covariates, high-dimensional clustering, or large-scale bootstrap procedures. The implementation exploits a layout convention—**ndarray**’s row-major storage appears as a transposed column-major matrix to cuBLAS—that avoids data reordering on the host side. Operations that are bandwidth-bound rather than compute-bound ($\mathbf{X}'\mathbf{y}$ via DGEMV) remain on CPU, where OpenBLAS is consistently faster. The feature is currently limited to NVIDIA GPUs via CUDA; extension to other backends (e.g., Vulkan compute, Metal) is left for future work.

6.2. Fully local deployment

While cloud-hosted language models offer convenience, many use cases require complete data isolation. The fully local configuration runs all components on the user’s machine via Ollama (Ollama Inc. 2024), eliminating all external network communication. Appendix D provides platform-specific installation instructions.

The evaluation results (Section 5) inform practical model recommendations. Open-source models can match proprietary alternatives: Qwen 2.5 72B achieves 100% accuracy on our benchmark, identical to Claude 3.5 Haiku. Users requiring complete data isolation need not sacrifice tool selection quality.

Table 9 summarizes model options with hardware requirements and recommended use cases. Ministral 3B demonstrates that even very small models achieve viable accuracy. With only 3 billion parameters, it runs on virtually any modern laptop—approximately 6 GB VRAM for GPU inference, or CPU-only with 8 GB RAM. The 9.2% error rate concentrates in genuinely ambiguous cases. The 7–8B parameter class offers a practical middle ground, fitting on consumer GPUs with 8+ GB VRAM (e.g., NVIDIA RTX 3070/4070) or Apple M-series devices with 16 GB unified memory.

Beyond LLM memory, users must budget RAM for dataset operations. A typical configuration analyzing 100,000 observations with a 7B model requires approximately 8–10 GB total RAM; a minimal laptop setup (Ministral 3B, small datasets) works within 16 GB. Appendix D.2

Table 9: Local deployment: models, hardware, and use cases

Model	Params	Storage	RAM	Acc. (%)	Recommended for
<i>High accuracy, workstation hardware</i>					
Qwen 2.5 72B	72B	42 GB	48 GB	100.0	Parameter-sensitive tasks
Llama 3.3 70B	70B	40 GB	48 GB	98.9	Multi-turn workflows
<i>Balanced accuracy, consumer hardware</i>					
Llama 3.1 8B	8B	4.7 GB	8 GB	— ^a	General use
Qwen 2.5 7B	7B	4.4 GB	8 GB	— ^a	General use
<i>Viable accuracy, minimal hardware</i>					
Minstral 3B	3B	2.0 GB	6 GB	90.8	Single-turn analyses

Note: Accuracy figures are from the 87-test single-turn benchmark (Section 5). Storage refers to Q4_K_M quantization (4-bit). RAM is memory during inference, typically 1.5–2× storage due to KV cache and computational buffers. ^a Not included in our benchmark; recommended based on external tool-calling evaluations (Berkeley Function-Calling Leaderboard). Multi-turn workflows benefit from 70B+ models that maintain context across iterative specification refinement.

provides detailed RAM requirements by dataset size (Table D.1) and system configurations for common deployment scenarios (Table D.2).

6.3. Privacy guarantees

The fully local configuration provides complete data isolation. No network requests are made to external services. All model inference occurs on the user’s hardware, dataset contents never leave the machine, and query history remains in local storage. This architecture is suitable for analysis of sensitive data where regulatory requirements (GDPR, HIPAA, or institutional review board protocols) or organizational policy prohibit cloud processing. Users should verify that their Ollama installation does not enable telemetry or automatic model updates. Appendix E provides detailed data transmission documentation and security best practices. The next section discusses limitations and future directions.

7. Discussion

This paper described chat-first data analytics—separating interpretation from computation, pre-validating methods rather than generating code, and keeping data local—and implemented the approach in **prompt2analytics**, a Rust library of over 200 validated methods exposed through MCP. The evaluation (Section 5) shows that tool selection works well: 90–100% accuracy on curated prompts, with errors concentrated in edge cases requiring fine distinctions between related methods. Several limitations qualify these results. The practical benefits of chat-first interfaces—accessibility, reduced cognitive load, exploratory dialogue—remain hypothesized, as no user studies were conducted. Controlled experiments comparing task completion time, error rates, and user satisfaction against traditional code-based workflows are needed and represent the most important direction for future research. When the correct tool is selected, manual inspection of recorded tool calls shows that core parameters are almost al-

ways correct; the main imprecision is omission of optional arguments that fall back on sensible defaults. Users should verify that the system ran the analysis they intended, not merely that it selected the right method. While method coverage is broad, important categories remain unimplemented—Bayesian inference, Heckman selection, Conley spatial standard errors, and structural equation modeling among them (Table A.1 in Appendix A documents gaps and recommends alternatives). Data are loaded entirely into memory, limiting applicability to datasets that fit in RAM. And because the MCP schema uses semantic versioning (currently 0.x), the API may change as the software matures. We follow a conservative deprecation policy: tool names and required parameters will not be removed or renamed without a major version increment; new optional parameters can be added in minor releases. Tool call transcripts from the examples in this paper will remain valid in all 0.x releases. The underlying MCP protocol (**rmcp** 0.8) is itself pre-1.0; protocol-level changes are absorbed by the server without affecting tool schemas.

These limitations suggest clear directions for future work. Reducing omission of optional parameters—through structured output constraints, two-pass verification where the model checks its own extraction before execution, or richer schema descriptions that make optional parameters more salient—would narrow the gap between correct tool selection and fully specified analysis execution. Property-based testing could strengthen correctness guarantees beyond comparison to reference implementations; QuickCheck-style testing for statistical properties (e.g., residuals orthogonal to regressors) would complement numerical validation. Foreign function interfaces to R and Python would let users call the Rust implementations from familiar environments, combining compiled performance with existing analytical workflows. An alternative design would have been to build **p2a-core** as an R package from the start, leveraging R’s ecosystem and user base. We chose a standalone architecture because the primary interface is the MCP server—a long-running process that any LLM client can connect to—which does not fit naturally into R’s package model. FFI wrappers remain a priority for giving R and Python users direct access to the compiled implementations. If extraction quality improves sufficiently, the bottleneck shifts from “did the model understand the request?” to “does the method library cover this analysis?”—at which point extending coverage becomes more valuable than improving the language model, which is exactly the division of labor the architecture is designed to exploit.

Even with these extensions, the system’s role would remain complementary. **prompt2analytics** is not a replacement for R, **Stata**, or **Python**. It occupies a different niche: users who know what analysis they want but not which software command produces it. A researcher fluent in R has no need for a conversational interface to OLS; a policy analyst who has never used R might. The system complements existing tools rather than competing with them—and for users who outgrow the conversational interface, every tool call maps to a public function that can be called directly from Rust or, once foreign function interfaces are available, from R and Python.

Maintaining a library of over 200 validated methods is a legitimate concern. The same AI-assisted development tools that motivate the project also address this: coding agents (e.g., Claude Code) can implement new methods, write validation tests, update dependencies, and respond to bug reports filed as GitHub issues—with human review of each pull request ensuring correctness. This workflow already produced the majority of the current implementation. It scales sublinearly with method count because new methods reuse established patterns (the **LinearEstimator** trait, shared matrix operations, the MCP handler template), and because

validation against R reference implementations is itself automatable. The project is open-source under the MIT license; community contributions follow the same review process.

More broadly, the chat-first approach exemplifies a pattern applicable beyond econometrics: using language models as interpreters between human intent and specialized software (Yao *et al.* 2023; Schick *et al.* 2023). Any domain where users express goals in natural language and systems execute well-defined operations—laboratory automation, geographic information systems, circuit simulation—could adopt the same architecture, particularly as structured protocols like MCP (Anthropic 2024) standardize the interface between models and tools.

The LLM landscape evolves rapidly. Models that achieve 100% on the current 87-test benchmark (Claude 3.5 Haiku, Qwen 2.5 72B) suggest the benchmark may already exhibit ceiling effects; expanding it with harder multi-turn and naturalistic cases is a priority. The evaluation harness and test cases are open-source, so new models can be benchmarked as they appear. Importantly, improving LLM capability does not undermine the architecture—it strengthens it. Better models reduce parameter omissions and may eventually make code-generation approaches more reliable, but the benefits of pre-validation and data locality are orthogonal to model quality. Even if future models generate flawless code, users who require auditable, reproducible analyses on local data would still benefit from fixed, validated tool implementations.

The success of this approach depends on a clear division of labor: language models interpret and explain; specialized software computes. Attempts to have language models do both—generating code that performs numerical computation—conflate interpretation with execution and sacrifice the reliability that scientific work requires (Schick *et al.* 2023). The viability of local LLM deployment shown in Section 6 has implications beyond data privacy. If small models can reliably select among pre-implemented methods, conversational interfaces to scientific software become possible without cloud connectivity or high-end hardware. The design of **prompt2analytics** suggests a template: implement algorithms in high-performance, memory-safe languages; expose them through structured protocols; and let language models handle the translation between human intent and computational execution.

Acknowledgments

The authors thank the reviewers for constructive feedback that improved the paper.

Appendix

Contents

- A Best practices and coverage gaps
- B LLM evaluation details
- C API response latency
- D Deployment guide
- E Privacy and security
- F Reproducibility
- G Benchmark methodology
- H Implementation notes
- I Extended examples

A. Best practices and coverage gaps

This appendix provides practical guidance for effective use of **prompt2analytics** and documents method coverage gaps.

A.1. Pre-analysis checklist

Before running an analysis through **prompt2analytics**, users should verify:

1. *Method availability*: Confirm the required method is implemented. Table A.1 documents coverage gaps; Appendix H provides method-specific caveats.
2. *Data requirements*: Verify the dataset meets method assumptions. Panel methods require entity and time identifiers; instrumental variables (IV) require instruments; difference-in-differences (DiD) requires treatment and time indicators.
3. *Variable naming*: Use clear, unambiguous column names. The LLM extracts variable names from natural language; names like `x1` or `var_1` are more error-prone than `wage` or `education`.
4. *Specification clarity*: Specify methods explicitly when precision matters. “Estimate a within-transformation fixed effects model with clustered standard errors at the firm level” yields more reliable results than “run a regression with firm effects.”

A.2. Recommended workflows

The system is most effective for:

- *Exploratory analysis*: Quickly testing specifications before committing to a formal analysis pipeline.
- *Teaching and learning*: Students can explore econometric methods conversationally while seeing which tools and parameters are invoked.

- *Rapid prototyping*: Researchers can iterate on model specifications in natural language before writing production code.
- *Verification*: Cross-checking results from other software by running the same analysis through an independent implementation.

Users should prefer R, Stata, or Python when methods are unavailable (Bayesian inference, Heckman selection), maximum performance is critical (**fixest** outperforms for high-dimensional fixed effects (HDFE) at scale), reproducibility requires code scripts, or publication-quality visualizations are needed.

A.3. Method coverage gaps

While method coverage is broad (over 200 methods), several important categories remain unimplemented. Table A.1 summarizes coverage gaps and recommends alternative tools.

Table A.1: Method coverage gaps

Method Category	Status	Recommended	Alternative
Bayesian inference	Not implemented	R: rstanarm , Python: PyMC	brms ;
Heckman selection	Not implemented	R: sampleSelection ;	Stata: heckman
Conley spatial SEs	Not implemented	R: fixest , lfe	
Full SARIMA (arbitrary seasonal)	Partial support	R: forecast	
Quantile IV	Not implemented	R: quantreg	
Structural equation modeling	Not implemented	R: lavaan	
Weak instrument inference	F -stat only; no Stock-Yogo CVs	R: ivmodel , AER	

Note: Lists unimplemented or partially implemented method categories with recommended alternatives in R, Stata, or Python.

Users encountering these gaps should consult the listed alternatives; future releases aim to address Bayesian inference and Heckman selection as priorities.

B. LLM evaluation details

This appendix provides detailed single-turn tool selection results from the evaluation described in Section 5.

B.1. Single-turn tool selection

Table B.1 presents accuracy, cost, and failure rates across all evaluated models. Table B.2 breaks down accuracy by test category.

The key pattern across both tables is that all models exceed 90% accuracy, with Ministral 3B the lowest at 90.8%. Accuracy drops concentrate in hypothesis testing and time series cate-

Table B.1: Tool selection accuracy and cost by model

Model	Size	Provider	N	Exact	Fail	Acc. (%)	Cost (\$)
Claude 3.5 Haiku	—	Anthropic	87	87	0	100.0	2.65
Qwen 2.5 72B	72B	OpenRouter	87	87	0	100.0	2.63
Nemotron 9B [†]	9B	OpenRouter	43	43	0	100.0	0.22
Llama 3.3 70B	70B	OpenRouter	87	86	0	98.9	1.17
DeepSeek [†]	—	OpenRouter	65	64	1	98.5	1.11
GPT-4o Mini	—	OpenAI	87	85	1	97.7	1.47
GPT-5 Nano	—	OpenAI	87	85	1	97.7	4.91
GPT-4.1 Nano	—	OpenAI	87	84	2	96.6	0.98
Minstral 3B	3B	OpenRouter	87	79	2	90.8	0.39
Qwen3 8B [†]	8B	OpenRouter	26	24	2	92.3	0.13

Note: [†] = partial evaluation due to API rate limits; interpret with caution. Cost estimates based on ~11,000 input tokens (tool schema) + 75 tokens (prompt/response) at January 2026 pricing. Local deployment has zero marginal API cost.

Table B.2: Accuracy by test category (%)

Category	Haiku	Qwen	Llama	4o-mini	5-nano	4.1-nano	Mini-3B
Regression	100.0	100.0	100.0	100.0	100.0	90.0	90.0
Panel Data	100.0	100.0	90.0	100.0	100.0	100.0	100.0
Causal Inference	100.0	100.0	100.0	100.0	100.0	100.0	90.9
Discrete Choice	100.0	100.0	100.0	90.0	100.0	80.0	100.0
Time Series	100.0	100.0	100.0	100.0	100.0	100.0	66.7
Hypothesis Testing	100.0	100.0	100.0	91.7	91.7	100.0	83.3
Machine Learning	100.0	100.0	100.0	100.0	91.7	100.0	100.0
Visualization	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Overall	100.0	100.0	98.9	97.7	97.7	96.6	90.8

Note: Each category contains 8–13 test cases covering basic to advanced use. Bold row shows overall accuracy across all 87 tests.

gories, where distinguishing between closely related methods (e.g., VAR vs. VECM) proves challenging. Cost per query varies by two orders of magnitude across providers.

C. API response latency

Figure C.1 presents response latency distributions for the models evaluated via their respective cloud APIs. These measurements include network round-trip time and reflect API-hosted inference, not local deployment performance.

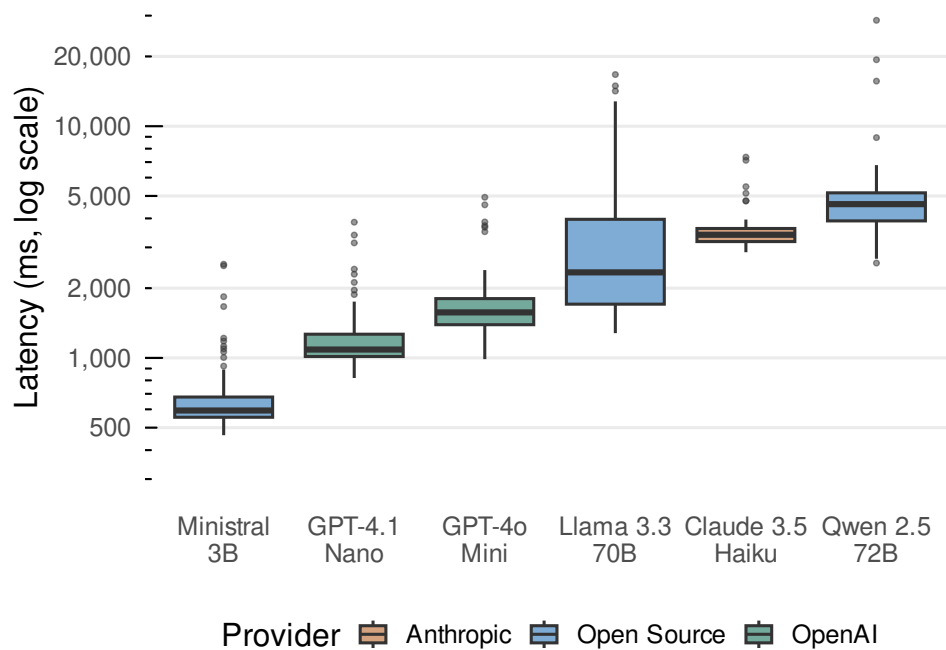


Figure C.1: API response latency by model (log scale). Latencies reflect cloud-hosted inference including network round-trip time; local deployment would differ substantially.

For cloud-hosted inference, GPT-4.1 Nano shows the fastest median response (1,245ms) among high-accuracy models, while Ministral 3B achieves 650ms median latency via OpenRouter. These figures are useful for comparing relative API performance but do not predict local inference speed.

Local deployment latency depends on several factors: model size and quantization level, available RAM and GPU VRAM, CPU or GPU compute capability, and context length (which affects KV cache size). As a rough guide, GPU-accelerated inference on consumer hardware (RTX 3070/4070 with 8 GB VRAM) typically achieves 1–3 seconds for 7–8B parameter models. CPU-only inference on modern laptops ranges from 5–15 seconds depending on model size and system specifications. For interactive use, GPU-accelerated inference with a 7–8B model provides sub-second tool selection, comparable to cloud API latency.

D. Deployment guide

Full platform-specific installation instructions (Linux, macOS, Windows/WSL2, Docker) are maintained in the repository’s README.md at <https://github.com/umatter/prompt2analytics>. Pre-compiled binaries for all major platforms are available on the GitHub releases page. This appendix documents only the MCP client configuration and hardware requirements referenced from Section 6.

D.1. MCP client configuration

For Claude Desktop, add to `claude_desktop_config.json`:

```
{
  "mcpServers": {
    "prompt2analytics": {
      "command": "/path/to/p2a-mcp",
      "args": ["--transport", "stdio"]
    }
  }
}
```

Other MCP-compatible clients (Cursor, Windsurf, custom agents) use the same `-transport stdio` interface. For HTTP-based integration, start the server with `-transport http -port 8080`.

D.2. Hardware requirements for data processing

Beyond LLM memory, users must budget RAM for dataset operations. Rust processes data in compact columnar format via **polars**. The primary memory consumers are design matrices ($8nk$ bytes for n observations and k predictors), covariance matrices ($8k^2$ bytes), bootstrap samples ($8kB$ bytes for B replications), and panel transformations (approximately equal in size to the original dataset).

Table D.1: RAM requirements for data processing

Rows	File size	Base RAM	Matrix ops	Notes
1,000	<1 MB	50 MB	+20 MB	Standard laptop
10,000	1–10 MB	100 MB	+100 MB	Standard laptop
100,000	10–100 MB	500 MB	+500 MB	Consumer hardware
1,000,000	100 MB–1 GB	2 GB	+2 GB	Workstation
10,000,000	1–10 GB	10 GB	+8 GB	High-memory system

Note: Peak memory during analysis. Assumes typical econometric datasets with 10–50 columns.

For combined LLM and data processing, sum the requirements from Tables 9 and D.1. A typical configuration analyzing 100,000 observations with a 7B model requires approximately 8–10 GB total RAM. Table D.2 provides system recommendations for common deployment scenarios.

Table D.2: System configurations by deployment scenario

Scenario	Model	RAM	GPU VRAM	Storage
Minimal (laptop)	Ministral 3B	16 GB	6 GB	5 GB
Consumer (desktop)	Qwen 2.5 7B	16 GB	8 GB	10 GB
Professional	Llama 3.3 70B	64 GB	48 GB	50 GB
Server (large data)	Qwen 2.5 72B	128 GB	2×24 GB	60 GB

Note: RAM includes both LLM and data processing headroom. GPU requirements assume CUDA-capable NVIDIA cards; AMD and Intel GPUs are supported via Ollama’s ROCm and OneAPI backends.

For the simplest setup, the Docker deployment (the Docker deployment subsection of Appendix D) bundles all dependencies including a pre-downloaded model and requires no local toolchain installation.

E. Privacy and security

This appendix documents data transmission behavior and security considerations.

E.1. Data transmission summary

Table E.1 lists operations that may transmit data beyond the local machine when using cloud LLM providers.

Table E.1: Data transmission by operation type

Operation	Data Transmitted	Risk Level
Dataset load	Column names, types, row count, null percentages	Low
Data preview (head/tail/sample)	Actual row values	High
Data profiling	Frequent values, min/max, examples	Medium
Statistical estimation	Coefficients, SEs, test statistics only	Low
Outlier detection	Specific observation values	Medium
Error messages	May include problematic values	Low

Note: Applies only when using cloud LLM providers. Local deployment via Ollama eliminates all external data transmission.

E.2. Audit logging

To enable audit logging of all MCP tool calls:

```
p2a-mcp --transport stdio --log-level debug --log-file audit.log
```

The log records: timestamp, tool name, parameters (excluding bulk data), and result summary. Review logs to verify no unexpected data transmission.

E.3. Network verification

To verify local-only operation with Ollama:

1. Start Ollama: `ollama serve`
2. Block outbound connections: `sudo ufw default deny outgoing`
3. Run analysis—it should complete successfully
4. Restore network: `sudo ufw default allow outgoing`

E.4. Security best practices

- Avoid loading untrusted data files (potential injection payloads)
- Review tool calls when processing sensitive data
- Use local LLM deployment for high-security environments
- Keep software updated; security patches are released as needed
- The MCP server has no filesystem access beyond loaded datasets

The primary mitigation for data privacy concerns is local LLM deployment via Ollama, which eliminates all external data transmission while maintaining full analytical capability.

F. Reproducibility

This appendix provides detailed instructions for reproducing the results in this paper.

F.1. Version information

Results in this paper were generated with the following software versions:

- Rust: 1.92.0 (stable)
- R: 4.5.2 with OpenBLAS
- Key R packages: **plm** 2.6-4, **lfe** 3.1-0, **sandwich** 3.1-1, **fixest** 0.12.1, **forecast** 8.23.0
- Operating system: Ubuntu 24.04 LTS (Linux 6.8)

F.2. Reproducing benchmarks

Performance benchmarks can be reproduced using the following protocol:

1. Install R 4.5+ with required packages: **plm**, **lfe**, **sandwich**, **forecast**, **bench**
2. Build **prompt2analytics** in release mode: `cargo build -release -p p2a-cli`
3. Execute benchmarks: `make -C paper/code benchmarks`

Both languages use `seed = 42` for random number generation. The benchmark protocol includes 10 warmup iterations before 100 measured iterations for fast methods (50 for moderate, 20 for slow). Detailed instructions: `paper/code/README_REPRODUCIBILITY.md`.

F.3. Containerized reproduction

The repository's `docker-compose.yml` provides a containerized deployment of the MCP server. For reproducing the paper's tables and figures, the native toolchain described above is recommended; a containerized paper build requires a multi-stage image with both R 4.5.2 (plus packages listed above) and Rust 1.92.0. A sample `Dockerfile` for this purpose is provided in `paper/Dockerfile.reproduce`; after building, run:

```
docker build -f paper/Dockerfile.reproduce -t p2a-paper .
docker run -v $(pwd)/paper:/output p2a-paper make all
```

Results should match within the tolerances specified in Appendix G.

G. Benchmark methodology

This appendix provides complete details on the benchmark methodology used in Section 6.

G.1. Data generating processes

Benchmark datasets are generated synthetically using identical data-generating processes in both languages with fixed random seeds (`seed = 42`) for exact reproducibility.

For regression benchmarks, data follow $y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \varepsilon_i$ with true parameters $\beta_0 = 1.0$, $\beta_1 = 2.0$, $\beta_2 = -1.5$, and $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ where $\sigma = 1.0$; covariates are drawn as $x_{1i} \sim \mathcal{N}(0, 1)$ and $x_{2i} \sim \mathcal{N}(0, 1)$ with correlation $\rho = 0.3$. For panel data, entity effects $\alpha_i \sim \mathcal{N}(0, 0.5^2)$ are added, with 100 entities observed over 10 periods. For discrete choice, the latent index $y_i^* = \beta'x_i + \varepsilon_i$ is transformed via the logit or probit link. For time series, data follow ARIMA(1, 1, 1) with $\phi = 0.7$, $\theta = 0.3$, and innovation variance $\sigma_\eta^2 = 1.0$.

Complete DGP specifications are provided in `paper/code/README_REPRODUCIBILITY.md`.

G.2. Benchmark execution protocol

Each benchmark configuration executes multiple iterations after a warmup phase:

- 100 iterations for fast methods (OLS, robust SEs, PCA)
- 50 iterations for moderate methods (panel FE, clustering)
- 20 iterations for slow methods (ARIMA, MSTL (multiple seasonal-trend decomposition using LOESS))

R benchmarks use the **bench** package, which accounts for garbage collection timing. Rust benchmarks use Criterion with automatic warmup and outlier detection. We report full timing distributions (minimum, quartiles, maximum, mean, standard deviation) rather than point estimates alone.

G.3. Hardware and software environment

All benchmarks run on identical hardware: an Intel Core i7-1260P processor with 64 GB RAM running Linux, with R 4.5.2 linked against OpenBLAS and Rust 1.92.0 compiled in release mode with link-time optimization. The **faer** library (version 0.22) used by **p2a-core** does *not* call an external BLAS; it implements its own GEMM microkernels in pure Rust with explicit SIMD via the **pulp** crate, dispatching at runtime to AVX2+FMA on x86-64 (or NEON on ARM). Both R and Rust therefore use optimized linear algebra routines, but from different implementations (OpenBLAS vs. **faer**). This makes the comparison a realistic measure of what each ecosystem delivers out of the box, not a controlled BLAS-versus-BLAS experiment.

G.4. Validation tolerances

Numerical validation against R reference implementations uses sample-size-dependent tolerances. We use absolute tolerances for quantities with known scale (e.g., test statistics) and relative tolerances for unbounded quantities (e.g., coefficient estimates).

Table G.1: Validation tolerances by method and sample size

Category	Quantity	Small ($n < 100$)	Medium	Large ($n > 10,000$)
<i>Coefficients and standard errors</i>				
OLS coefficients	Relative	10^{-6}	10^{-8}	10^{-10}
Robust SEs (HC0/HC1) ^a	Relative	10^{-6}	10^{-8}	10^{-8}
Robust SEs (HC2/HC3) ^a	Relative	10^{-5}	10^{-7}	10^{-7}
Panel FE/RE	Relative	10^{-5}	10^{-6}	10^{-6}
MLE ^b (logit/probit)	Relative	10^{-4}	10^{-6}	10^{-6}
<i>Test statistics and fit measures</i>				
t -statistics, F -statistics	Absolute	10^{-4}	10^{-6}	10^{-6}
R^2 , within R^2	Absolute	10^{-4}	10^{-6}	10^{-6}
p -values	Absolute	10^{-4}	10^{-6}	10^{-6}
Log-likelihood	Absolute	10^{-2}	10^{-4}	10^{-4}
<i>Hypothesis tests</i>				
Test statistics	Absolute	10^{-4}	10^{-6}	10^{-6}
Critical values	Absolute	10^{-4}	10^{-4}	10^{-4}

Note: Absolute tolerances (ε_{abs}) apply to test statistics, p -values, and bounded quantities. Relative tolerances (ε_{rel}) apply to coefficient estimates and standard errors. ^a HC0–HC3: heteroskedasticity-consistent standard error estimators (types 0–3). ^b MLE: maximum likelihood estimation.

Looser tolerances for small samples reflect increased sensitivity to algorithmic differences when matrices are ill-conditioned. HC2/HC3 standard errors use looser tolerances because leverage calculation involves $h_{ii} = x_i'(X'X)^{-1}x_i$, which accumulates rounding errors differently depending on whether leverage is extracted from QR decomposition (as in **sandwich**) or computed via direct matrix multiplication (as in **prompt2analytics**). Both approaches are numerically valid; the tolerance reflects implementation variation rather than error.

MLE methods use looser small-sample tolerances because convergence criteria differ between implementations: R's **glm()** uses relative change in deviance, while **prompt2analytics** uses absolute change in log-likelihood. Final parameter estimates can differ at the tolerance boundary

while both satisfying their respective convergence criteria.

Tolerances were determined empirically by running validation tests across sample sizes from $n = 10$ to $n = 100,000$ and identifying the tightest tolerance that passes consistently. Edge cases (singular matrices, near-separation in logit/probit, small cluster counts) are tested separately with method-specific tolerances documented in the test code. All 334 validation tests pass at the specified tolerances.

H. Implementation notes

This appendix documents implementation caveats and limitations for specific methods.

H.1. Instrumental variables

The two-stage least squares (2SLS) implementation reports first-stage F -statistics for instrument relevance. The traditional rule of thumb ($F > 10$) from [Staiger and Stock \(1997\)](#) has been superseded by more nuanced guidance. [Stock and Yogo \(2005\)](#) provide critical values depending on the number of instruments and acceptable bias levels; [Lee, McCrary, Moreira, and Porter \(2022\)](#) offer further refinements for inference under weak identification. The current implementation does not provide Stock-Yogo critical values or weak-identification-robust inference methods (Anderson-Rubin, conditional likelihood ratio). Users with potentially weak instruments should verify results using R's **ivmodel** or **AER** packages.

H.2. Difference-in-differences

The difference-in-differences (DiD) implementation assumes the canonical two-group, two-period design. Recent econometric literature has documented severe biases when this estimator is applied to staggered treatment timing. [Goodman-Bacon \(2021\)](#) demonstrates that two-way fixed effects DiD produces a weighted average of all possible 2×2 comparisons, including problematic “forbidden” comparisons using already-treated units as controls. [de Chaisemartin and D’Haultfoeuille \(2020\)](#) and [Sun and Abraham \(2021\)](#) propose alternative estimators robust to treatment effect heterogeneity.

prompt2analytics implements staggered DiD via Callaway-Sant’Anna ([Callaway and Sant’Anna 2021](#)), Goodman-Bacon decomposition, and extended two-way fixed effects (TWFE) ([Wooldridge 2021](#)). Users with complex staggered designs may also consult **fixest**’s `sunab()` function or the **did** package in R for additional diagnostics.

H.3. Random effects

The random effects estimator uses the Swamy-Arora variance component estimator, which can produce negative variance estimates for σ_α^2 (the between-entity variance) in certain data configurations. When this occurs, the implementation truncates the estimate to zero and proceeds with pooled OLS, issuing a warning.

H.4. Discrete choice models

Logit and probit models report coefficients on the latent index scale. For interpretation, marginal effects are computed as *Average Marginal Effects* (AME): the partial derivative

$\partial \Pr(y = 1)/\partial x_j$ is evaluated at each observation’s covariate values, then averaged across the sample. This differs from *Marginal Effects at Means* (MEM), which evaluates derivatives at the sample mean of all covariates. AME is generally preferred because it does not depend on potentially unrepresentative “average” covariate values (Cameron and Trivedi 2005).

H.5. Time series models

Autoregressive integrated moving average (ARIMA) estimation supports non-seasonal models: $\text{ARIMA}(p, d, q)$ with automatic or user-specified orders. Seasonal ARIMA (SARIMA) is partially implemented: seasonal differencing and seasonal AR/MA terms can be specified, but automatic seasonal order selection is not available.

Vector autoregression (VAR) and vector error correction model (VECM) implementations support lag selection via AIC/BIC and Johansen cointegration testing, but do not implement structural VAR identification.

H.6. Regression diagnostics validation

Diagnostic tests are validated against **lmtest** (Hothorn, Zeileis, Farebrother, and Cummins 2024): Jarque-Bera normality test statistics match to $< 10^{-4}$, Breusch-Pagan heteroskedasticity tests to $< 10^{-4}$, Durbin-Watson autocorrelation statistics to $< 10^{-6}$, and variance inflation factors (VIF) to $< 10^{-6}$.

H.7. Known differences from reference implementations

A small number of intentional differences exist between **prompt2analytics** and reference implementations. First, **prompt2analytics** defaults to HC1 robust standard errors (consistent with Stata’s **robust** option), while R’s **sandwich** defaults to HC3. Second, R uses variable names in output; **prompt2analytics** uses numeric indices internally but displays names in formatted output. Third, for panel models we report “within R^2 ” by default, matching **plm**’s convention for fixed effects models. Fourth, iterative methods (HDFE, maximum likelihood estimation) use a default convergence tolerance of 10^{-8} , which may differ slightly from R package defaults. These differences are documented in the method documentation and do not affect the validity of statistical inference.

Coverage gaps and unimplemented methods are documented in Appendix A (Table A.1).

I. Extended examples

This appendix collects additional examples that complement the core demonstrations in Section 4.

I.1. Comparison with R

For reference, the equivalent R code for the fixed effects estimation shown in Section 4.1 is:

```
library(plm)
data("Grunfeld", package = "plm")
fe_model <- plm(inv ~ value + capital, data = Grunfeld,
```

```

index = c("firm", "year"), model = "within")
summary(fe_model)

```

The **prompt2analytics** implementation produces numerically equivalent results (Section 5), while offering a conversational interface for users unfamiliar with R syntax.

I.2. Time series analysis

A time series workflow using quarterly U.S. GDP growth data (1954–2023) illustrates method progression within a single conversation: stationarity testing, univariate modeling, then multivariate analysis.

User: Load gdp_quarterly.csv and check for stationarity.

```

**Augmented Dickey-Fuller Test** (gdp_growth)
Statistic: -4.82, P-value: 0.000
GDP growth is stationary. No differencing needed.

```

User: Fit an ARIMA model to GDP growth. Let the system pick the order.

```

**ARIMA(2,0,1)** (AIC-based selection)
AR(1): 0.312 (0.067), AR(2): 0.154 (0.063), MA(1): -0.089 (0.072)
Ljung-Box (lag 10): p = 0.482 (no residual autocorrelation)

```

Order selection is delegated to the LLM, which chooses p , d , q based on the ADF result and its training knowledge. This is not equivalent to R's `auto.arima()`, which performs systematic grid search (Hyndman and Khandakar 2008).

I.3. Handling messy data

Real-world datasets often require cleaning before analysis. The system detects non-standard missing indicators and guides type conversion:

User: Load survey_responses.csv and describe the data.

```

Dataset: survey_responses (2,847 rows x 6 columns)
Note: 'income' and 'satisfaction' are typed as String, likely due to
non-standard missing value indicators (".", "N/A").

```

User: Replace "." and "N/A" with proper missing values, then convert income and satisfaction to numeric.

```

Cleaned: 89 values replaced across 2 columns.
income:      now Float64, 52 missing (1.8%)
satisfaction: now Float64, 37 missing (1.3%)

```

References

- Abadie A, Diamond A, Hainmueller J (2010). “Synthetic Control Methods for Comparative Case Studies: Estimating the Effect of California’s Tobacco Control Program.” *Journal of the American Statistical Association*, **105**(490), 493–505. doi:[10.1198/jasa.2009.ap08746](https://doi.org/10.1198/jasa.2009.ap08746).
- Anselin L (1988). *Spatial Econometrics: Methods and Models*. Springer, Dordrecht. doi:[10.1007/978-94-015-7799-1](https://doi.org/10.1007/978-94-015-7799-1).
- Anthropic (2024). “Model Context Protocol.” Accessed: 2024, URL <https://modelcontextprotocol.io/>.
- Arellano M, Bond S (1991). “Some Tests of Specification for Panel Data: Monte Carlo Evidence and an Application to Employment Equations.” *Review of Economic Studies*, **58**(2), 277–297. doi:[10.2307/2297968](https://doi.org/10.2307/2297968).
- Austin PC (2011). “An Introduction to Propensity Score Methods for Reducing the Effects of Confounding in Observational Studies.” *Multivariate Behavioral Research*, **46**(3), 399–424. doi:[10.1080/00273171.2011.568786](https://doi.org/10.1080/00273171.2011.568786).
- Baltagi BH (2013). *Econometric Analysis of Panel Data*. 5th edition. Wiley, Chichester.
- Bergé L (2018). “Efficient Estimation of Maximum Likelihood Models with Multiple Fixed-Effects: The R Package **fixest**.” *Journal of Statistical Software*, **106**(3), 1–35. doi:[10.18637/jss.v106.i03](https://doi.org/10.18637/jss.v106.i03).
- Callaway B, Sant’Anna PHC (2021). “Difference-in-Differences with Multiple Time Periods.” *Journal of Econometrics*, **225**(2), 200–230. doi:[10.1016/j.jeconom.2020.12.001](https://doi.org/10.1016/j.jeconom.2020.12.001).
- Calonico S, Cattaneo MD, Titiunik R (2014). “Robust Nonparametric Confidence Intervals for Regression-Discontinuity Designs.” *Econometrica*, **82**(6), 2295–2326. doi:[10.3982/ECTA11757](https://doi.org/10.3982/ECTA11757).
- Cameron AC, Trivedi PK (2005). *Microeconometrics: Methods and Applications*. Cambridge University Press, Cambridge. doi:[10.1017/CB09780511811241](https://doi.org/10.1017/CB09780511811241).
- Carpenter B, Gelman A, Hoffman MD, Lee D, Goodrich B, Betancourt M, Brubaker M, Guo J, Li P, Riddell A (2017). “Stan: A Probabilistic Programming Language.” *Journal of Statistical Software*, **76**(1), 1–32. doi:[10.18637/jss.v076.i01](https://doi.org/10.18637/jss.v076.i01).
- Cattaneo MD, Feng Y, Titiunik R (2021). “Prediction Intervals for Synthetic Control Methods.” *Journal of the American Statistical Association*, **116**(536), 1865–1880. doi:[10.1080/01621459.2021.1979561](https://doi.org/10.1080/01621459.2021.1979561).
- Chambers JM (1998). *Programming with Data: A Guide to the S Language*. Springer-Verlag, New York. doi:[10.1007/978-1-4684-6310-8](https://doi.org/10.1007/978-1-4684-6310-8).
- Chambers JM, Hastie TJ (1992). *Statistical Models in S*. Wadsworth & Brooks/Cole, Pacific Grove, CA.

- Chen S, Shen K, Ma X (2025). “Econometrics AI Agent.” *arXiv preprint arXiv:2506.00856*. URL <https://arxiv.org/abs/2506.00856>.
- Chernozhukov V, Chetverikov D, Demirer M, Duflo E, Hansen C, Newey W, Robins J (2018). “Double/Debiased Machine Learning for Treatment and Structural Parameters.” *Econometrics Journal*, **21**(1), C1–C68. doi:10.1111/ectj.12097.
- Cinelli C, Hazlett C (2020). “Making Sense of Sensitivity: Extending Omitted Variable Bias.” *Journal of the Royal Statistical Society: Series B*, **82**(1), 39–67. doi:10.1111/rssb.12348.
- Codd EF (1974). “Seven Steps to Rendezvous with the Casual User.” *IFIP Working Conference on Data Base Management*, pp. 179–200.
- Croissant Y, Milla G (2008). “Panel Data Econometrics in R: The **plm** Package.” *Journal of Statistical Software*, **27**(2), 1–43. doi:10.18637/jss.v027.i02.
- Davidson R, MacKinnon JG (2004). *Econometric Theory and Methods*. Oxford University Press, New York.
- de Chaisemartin C, D’Haultfoeuille X (2020). “Two-Way Fixed Effects Estimators with Heterogeneous Treatment Effects.” *American Economic Review*, **110**(9), 2964–2996. doi:10.1257/aer.20181169.
- Feurer M, Klein A, Eggenberger K, Springenberg JT, Blum M, Hutter F (2015). “Efficient and Robust Automated Machine Learning.” In *Advances in Neural Information Processing Systems 28 (NeurIPS)*, pp. 2962–2970.
- Gaure S (2013). “**lfe**: Linear Group Fixed Effects.” *The R Journal*, **5**(2), 104–117. URL <https://journal.r-project.org/archive/2013/RJ-2013-031/>.
- Goodman-Bacon A (2021). “Difference-in-Differences with Variation in Treatment Timing.” *Journal of Econometrics*, **225**(2), 254–277. doi:10.1016/j.jeconom.2021.03.014.
- Google (2025). “Agent2Agent Protocol (A2A).” Announced April 2025; transferred to Linux Foundation, URL <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>.
- Greene WH (2018). *Econometric Analysis*. 8th edition. Pearson, New York.
- Hamilton JD (1994). *Time Series Analysis*. Princeton University Press, Princeton, NJ.
- Hansen LP (1982). “Large Sample Properties of Generalized Method of Moments Estimators.” *Econometrica*, **50**(4), 1029–1054. doi:10.2307/1912775.
- Ho DE, Imai K, King G, Stuart EA (2011). “**MatchIt**: Nonparametric Preprocessing for Parametric Causal Inference.” *Journal of Statistical Software*, **42**(8), 1–28. doi:10.18637/jss.v042.i08.
- Hollmann N, Müller S, Hutter F (2024). “Large Language Models for Automated Machine Learning.” In *Proceedings of the 2024 Conference on Neural Information Processing Systems (NeurIPS)*. URL <https://arxiv.org/abs/2408.14033>.

- Hong S, Lin Y, Liu B, Liu B, Wu B, Li D, Chen J, Zhang J, Wang J, Zhang L, Zhang L, Yang M, Zhuge M, Guo T, Zhou T, Tao W, Wang W, Tang X, Lu X, Zheng X, Liang X, Fei Y, Cheng Y, Xu Z, Wu C (2025). “Data Interpreter: An LLM Agent For Data Science.” In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 19796–19821. URL <https://arxiv.org/abs/2402.18679>.
- Hothorn T, Zeileis A, Farebrother RW, Cummins C (2024). *lmtest: Testing Linear Regression Models*. R package version 0.9-40, URL <https://CRAN.R-project.org/package=lmtest>.
- Hu K (2023). “ChatGPT Sets Record for Fastest-Growing User Base.” Reuters. Accessed January 2026, URL <https://www.reuters.com/technology/chatgpt-sets-record-fastest-growing-user-base-analyst-note-2023-02-01/>.
- Hyndman RJ, Khandakar Y (2008). “Automatic Time Series Forecasting: The **forecast** Package for R.” *Journal of Statistical Software*, **27**(3), 1–22. doi:10.18637/jss.v027.i03.
- IBM Research (2025). “Agent Communication Protocol (ACP).” Launched March 2025; merged with A2A under Linux Foundation, URL <https://research.ibm.com/projects/agent-communication-protocol>.
- Imai K, Ratkovic M (2014). “Covariate Balancing Propensity Score.” *Journal of the Royal Statistical Society: Series B*, **76**(1), 243–263. doi:10.1111/rssb.12027.
- Koenker R, Jr GB (1978). “Regression Quantiles.” *Econometrica*, **46**(1), 33–50. doi:10.2307/1913643.
- LeDell E, Poirier S (2020). “H2O AutoML: Scalable Automatic Machine Learning.” In *Proceedings of the 7th ICML Workshop on Automated Machine Learning*. URL https://www.automl.org/wp-content/uploads/2020/07/AutoML_2020_paper_61.pdf.
- Lee DS, McCrary J, Moreira MJ, Porter J (2022). “Valid *t*-ratio Inference for IV.” *American Economic Review*, **112**(10), 3260–3290. doi:10.1257/aer.20211063.
- Li B, Luo Y, Chai C, Li G, Tang N (2024). “The Dawn of Natural Language to SQL: Are We Fully Ready?” *Proceedings of the VLDB Endowment*, **17**(11), 3318–3331. doi:10.14778/3681954.3682024.
- Liu W, Huang X, Zeng X, Hao X, Yu S, Li D, Wang S, Gan W, Liu Z, Yu Y, Wang Z, Wang Y, Ning W, Hou Y, Wang B, Wu C, Wang X, Liu Y, Wang Y, Tang D, Tu D, Shang L, Jiang X, Tang R, Lian D, Liu Q, Chen E (2025). “ToolACE: Winning the Points of LLM Function Calling.” In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. URL <https://openreview.net/forum?id=8EB8k6DdCU>.
- Longley JW (1967). “An Appraisal of Least Squares Programs for the Electronic Computer from the Point of View of the User.” *Journal of the American Statistical Association*, **62**(319), 819–841. doi:10.1080/01621459.1967.10500896.
- Ludwig J, Mullainathan S, Rambachan A (2025). “Large Language Models: An Applied Econometric Framework.” *NBER Working Paper*, (33344). doi:10.3386/w33344. URL <https://www.nber.org/papers/w33344>.

- McCullough BD (1997). “Assessing the Reliability of Statistical Software: Part I.” *The American Statistician*, **52**(4), 358–366. doi:10.2307/2685442.
- McCullough BD, Vinod HD (1999). “The Numerical Reliability of Econometric Software.” *Journal of Economic Literature*, **37**(2), 633–665. doi:10.1257/jel.37.2.633.
- Ollama Inc (2024). *Ollama: Run Large Language Models Locally*. URL <https://ollama.ai>.
- Pola-rs Contributors (2024). *polars: Blazingly Fast DataFrames in Rust and Python*. Version 0.46, URL <https://pola.rs/>.
- Qu C, Dai S, Wei X, Xu J, Cai H, Wang S, Yin D, Wen JR (2025). “Tool Learning with Language Models: A Comprehensive Survey of Methods, Pipelines, and Benchmarks.” *Vic-inagearth*. doi:10.1007/s44336-025-00024-x.
- Renggli C, Ilyas IF, Rekatsinas T (2025). “Fundamental Challenges in Evaluating Text2SQL Solutions and Detecting Their Limitations.” *arXiv preprint*. ArXiv:2501.18197, URL <https://arxiv.org/abs/2501.18197>.
- Schick T, Dwivedi-Yu J, Dessì R, Raileanu R, Lomeli M, Zettlemoyer L, Cancedda N, Scialom T (2023). “Toolformer: Language Models Can Teach Themselves to Use Tools.” In *Advances in Neural Information Processing Systems (NeurIPS)*. URL <https://arxiv.org/abs/2302.04761>.
- Staiger D, Stock JH (1997). “Instrumental Variables Regression with Weak Instruments.” *Econometrica*, **65**(3), 557–586. doi:10.2307/2171753.
- Stock JH, Yogo M (2005). “Testing for Weak Instruments in Linear IV Regression.” In DWK Andrews, JH Stock (eds.), *Identification and Inference for Econometric Models: Essays in Honor of Thomas Rothenberg*, pp. 80–108. Cambridge University Press, Cambridge. doi:10.1017/CB09780511614491.006.
- Sun L, Abraham S (2021). “Estimating Dynamic Treatment Effects in Event Studies with Heterogeneous Treatment Effects.” *Journal of Econometrics*, **225**(2), 175–199. doi:10.1016/j.jeconom.2020.09.006.
- Sun M, Han R, Jiang B, Qi H, Sun D, Yuan Y, Huang J (2025). “LAMBDA: A Large Model Based Data Agent.” *Journal of the American Statistical Association*. doi:10.1080/01621459.2025.2510000. Online first.
- van der Laan MJ, Rose S (2011). *Targeted Learning: Causal Inference for Observational and Experimental Data*. Springer, New York. doi:10.1007/978-1-4419-9782-1.
- Wilkinson GN, Rogers CE (1973). “Symbolic Description of Factorial Models for Analysis of Variance.” *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, **22**(3), 392–399. doi:10.2307/2346786.
- Woods WA (1973). “Progress in Natural Language Understanding: An Application to Lunar Geology.” In *Proceedings of the AFIPS National Computer Conference*, pp. 441–450. doi:10.1145/1499586.1499695.

- Wooldridge JM (2010). *Econometric Analysis of Cross Section and Panel Data*. 2nd edition. MIT Press, Cambridge, MA.
- Wooldridge JM (2021). “Two-Way Fixed Effects, the Two-Way Mundlak Regression, and Difference-in-Differences Estimators.” *Econometrica*. doi:10.2139/ssrn.3906345. Forthcoming; originally circulated as SSRN Working Paper 3906345.
- Xu Y (2017). “Generalized Synthetic Control Method: Causal Inference with Interactive Fixed Effects Models.” *Political Analysis*, 25(1), 57–76. doi:10.1017/pan.2016.2.
- Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y (2023). “ReAct: Synergizing Reasoning and Acting in Language Models.” In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. URL <https://arxiv.org/abs/2210.03629>.
- Yu T, Zhang R, Yang K, Yasunaga M, Wang D, Li Z, Ma J, Li I, Yao Q, Roman S, Zhang Z, Radev D (2018). “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task.” In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3911–3921. doi:10.18653/v1/D18-1425.
- Zeileis A (2004). “Econometric Computing with HC and HAC Covariance Matrix Estimators.” *Journal of Statistical Software*, 11(10), 1–17. doi:10.18637/jss.v011.i10.

Affiliation:

Ulrich Matter
 Bern University of Applied Sciences, Bern, Switzerland
 University of St. Gallen, St. Gallen, Switzerland
 E-mail: ulrich.matter@unisg.ch
 URL: <https://umatter.github.io/>